DevOps Laboratory Manual

Table of Contents

Preface

This manual covers twelve experiments that introduce the core tools and practices of DevOps: web development, version control, continuous integration and deployment, containerization, container orchestration, and automated browser testing. Each experiment builds on the one before it. The event registration application built in Experiment 01 reappears throughout the series as the subject of version control, CI/CD pipelines, Docker images, Kubernetes deployments, and Selenium test cases.

Platform note: All experiments are written for an Ubuntu/Debian-based Linux environment. Students working on Windows should use WSL2 (Windows Subsystem for Linux 2) and run commands from a WSL2 terminal. See the WSL2 setup note below.

Setting up WSL2 on Windows: If you are on Windows and have not yet set up WSL2, do this before starting Experiment 01.

1. Open **PowerShell** as Administrator (right-click the Start menu → *Terminal (Admin)* or *Windows PowerShell (Admin)*).
2. Run: `wsl --install` This installs WSL2 and Ubuntu automatically. If prompted, restart your computer.
3. After restarting, Ubuntu opens and asks you to create a Linux username and password — these are separate from your Windows credentials. Choose any username and remember the password; it is used for `sudo` commands throughout the experiments.
4. Once you see the Ubuntu shell prompt (e.g., `username@DESKTOP:~$`), WSL2 is ready.
5. Update the Ubuntu package list before starting any experiment: `bash sudo apt update && sudo apt upgrade -y`

All experiment commands should be typed into this Ubuntu (WSL2) terminal, not into the Windows Command Prompt or PowerShell.

Instructor Reference Repository: A public repository demonstrating the expected file structure, commits, and content for all twelve experiments is available at `https://github.com/ManishGantyala/btech-devops-labs`. Students may inspect it at any point to verify that their own work is on the right track; forking it to keep a personal copy in your own GitHub account is also fine. A fork of this repository is not a substitute for each student's own working repository — each student creates and maintains their own GitHub repository as part of the practical work in Experiments 02 onward.

Experiment 01 — Event Registration Web Application

Aim

To write code for a simple user registration form for an event.

Learning Objectives

- Create a simple user registration form for an event.
- Design the registration form using HTML and CSS.
- Provide input fields for user registration details.
- Use JavaScript for client-side input handling.
- Display a registration success message after form submission.

Requirements

- A text editor (any plain text editor will work).
- A web browser (Chrome, Firefox, or Edge).
- No installation is required for this experiment.

Concept

What HTML, CSS, and JavaScript Each Do

A web page is built from three technologies, each handling a different concern:

Technology	File	Role
HTML	`index.html`	Defines the **structure** — the elements on the page: headings, labels, input fields, buttons
CSS	`style.css`	Defines the **appearance** — colours, font sizes, spacing, borders, layout
JavaScript	`script.js`	Defines the **behaviour** — what the page does when a user types or clicks

The browser loads all three files together. `index.html` connects to the other two:

```
<link rel="stylesheet" href="style.css">   <!-- loads the CSS -->
<script src="script.js"></script>          <!-- loads the JavaScript -->
```

Why a Web Form?

A web form enforces structure — every submission provides the same fields in the same order. The browser's built-in input types (`email`, `tel`, `required`) catch formatting mistakes immediately. JavaScript adds a second layer by filtering invalid characters from the name and phone fields as the user types.

What "Client-Side" Means

This application is **client-side only**: all logic runs inside the visitor's browser. There is no server and no database. When the form is submitted, `script.js` calls `event.preventDefault()`, which stops the browser's default form-submission behaviour (which would normally send data to a server) and instead shows the success message locally. The form works by opening `index.html` directly as a local file — no installation or running server is required.

Project Structure

```
experiment-01/
├── index.html
├── style.css
├── script.js
└── README.md
```

Application Description

The application is a TechFest 2026 event registration form with the following fields:

- Full Name
- Email
- Phone Number
- Department
- Year of Study
- Event Selection

JavaScript is used to: - Restrict the Full Name field to alphabetic characters and spaces. - Restrict the Phone Number field to numeric characters only. - Display a "Registration successful!" message when the form is submitted.

Source Code

index.html

```
<!DOCTYPE html>
<html>

<head>
    <title>TechFest 2026 - Event Registration</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>
    <h1>TechFest 2026 - Event Registration</h1>
    <h2>Register for the upcoming events</h2>

    <div class="form-container">

        <form id="registrationForm">

            <label for="name">Full Name:</label>
            <input
                type="text"
                id="name"
                name="name"
                required
            >

            <label for="email">Email:</label>
            <input
                type="email"
                id="email"
                name="email"
                required
            >

            <label for="phone">Phone Number:</label>
            <input
                type="tel"
                id="phone"
                name="phone"
                maxlength="10"
                inputmode="numeric"
                required
            >

            <label for="department">Department:</label>
            <select id="department" name="department" required>
                <option value="">Select Department</option>
                <option value="cse">Computer Science and Engineering</option>
                <option value="ece">Electronics and Communication Engineering</option>
                <option value="eee">Electrical and Electronics Engineering</option>
                <option value="mech">Mechanical Engineering</option>
                <option value="civil">Civil Engineering</option>
            </select>
```

```
            <label for="year">Year of Study:</label>
            <select id="year" name="year" required>
                <option value="">Select Year</option>
                <option value="1">First Year</option>
                <option value="2">Second Year</option>
                <option value="3">Third Year</option>
                <option value="4">Fourth Year</option>
            </select>

            <label for="event">Select Event:</label>
            <select id="event" name="event" required>
                <option value="">Select Event</option>
                <option value="web-development">
                    Web Development Workshop
                </option>
                <option value="cloud-computing">
                    Cloud Computing Workshop
                </option>
                <option value="ai-ml">
                    AI and Machine Learning Seminar
                </option>
            </select>

            <button type="submit">Register</button>

        </form>

        <p id="message"></p>
    </div>

    <script src="script.js"></script>
</body>

</html>
```

style.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
}

h1 {
    text-align: center;
    margin-top: 30px;
}

h2 {
    text-align: center;
    margin-bottom: 20px;
}

.form-container {
    width: 400px;
    margin: 40px auto;
    padding: 20px;
```

```
    background-color: white;
    border-radius: 8px;
    border: 1px solid #ddd;
}

.form-container label {
    display: block;
    margin-bottom: 6px;
    font-weight: bold;
}

.form-container input {
    width: 100%;
    padding: 10px;
    margin-bottom: 15px;
    box-sizing: border-box;
}

.form-container select {
    width: 100%;
    padding: 10px;
    margin-bottom: 15px;
    box-sizing: border-box;
}

.form-container button {
    width: 100%;
    padding: 10px;
    margin-top: 10px;
    cursor: pointer;
}

.form-container button:hover {
    opacity: 0.9;
}

.form-container input:focus,
.form-container select:focus {
    outline: 2px solid #333;
}
```

script.js

```
const nameInput = document.getElementById("name");
const phoneInput = document.getElementById("phone");

nameInput.addEventListener("input", function () {
    this.value = this.value.replace(/[^A-Za-z ]/g, "");
});

phoneInput.addEventListener("input", function () {
    this.value = this.value.replace(/[^0-9]/g, "");
});

const registrationForm = document.getElementById("registrationForm");
const message = document.getElementById("message");
```

```
registrationForm.addEventListener("submit", function (event) {
    event.preventDefault();

    message.textContent = "Registration successful!";
});
```

Procedure

Step 1 — Open the Experiment Directory

The experiment files are in the `experiment-01/` directory. Three files are present: `index.html`, `style.css`, and `script.js`.

Step 2 — Review the HTML File

Open `index.html` in a text editor. Each `<label>` paired with an `<input>` or `<select>` corresponds to one field on the registration form. The `id` attribute on each field is what JavaScript uses to locate and interact with that element.

Step 3 — Review the CSS File

Open `style.css` in a text editor. The `.form-container` class and its nested selectors control the appearance of the form box, labels, inputs, and button.

Step 4 — Review the JavaScript File

Open `script.js` in a text editor. Observe three event listeners: - One on the name input — removes non-letter characters on every keystroke. - One on the phone input — removes non-digit characters on every keystroke. - One on the form's `submit` event — `event.preventDefault()` stops the browser from sending a network request; `message.textContent` sets the confirmation text.

Step 5 — Observe How the Files Are Linked

The HTML file connects to the CSS in the `<head>` section:

```
<link rel="stylesheet" href="style.css">
```

and to the JavaScript at the bottom of `<body>`:

```
<script src="script.js"></script>
```

Placing the `<script>` tag at the bottom of `<body>` ensures all HTML elements exist before JavaScript tries to find them by their `id`.

Step 6 — Open the Application in a Browser

Open `index.html` directly in a web browser — no web server is required.

Option 1 — File manager: Navigate to the `experiment-01/` folder and double-click `index.html`.

Option 2 — Terminal (Linux / WSL):

```
xdg-open experiment-01/index.html
```

Observe: The TechFest 2026 Event Registration form loads in the browser, showing all six fields and the Register button.

Verification

Check	How to verify	Expected result
Form loads correctly	Open the page	All six fields and the Register button are visible
Name filtering	Type `"Test123 Student!"` into Full Name	Field shows only `"Test Student"` — digits and `!` removed immediately
Phone filtering	Type `"98765abc"` into Phone Number	Field shows only `"98765"` — letters removed immediately
Form submission	Fill all fields and click Register	`"Registration successful!"` appears below the form
Required field validation	Leave a field empty and click Register	Browser highlights the empty field and blocks submission; no success message appears

Result

The simple user registration form for the TechFest 2026 event was developed using HTML, CSS, and JavaScript.

Experiment 02 — Explore Git and GitHub Commands

Aim

To explore Git and GitHub commands, and to understand how they work together to track changes in a project and collaborate using a remote repository.

Learning Objectives

- Explain why version control is needed.
- Explain what Git is and what GitHub is, and how they differ.
- Create a GitHub account and configure Git locally.
- Track a file through the working directory → staging area → commit cycle.
- View commit history and inspect changes.
- Create, switch, and merge branches.
- Create a personal GitHub repository, add the Experiment 01 application to it, and push, pull, and clone it.

Requirements

- A GitHub account (free). If you do not have one yet, go to `https://github.com`, click **Sign up**, and create an account before continuing. You will need it for Step 14 onward.
- A computer with Git installed. Install it if needed:

```
sudo apt update
sudo apt install git -y
```

- A terminal or command prompt.
- The three Experiment 01 files (`index.html`, `style.css`, `script.js`) that you built in Experiment 01.

Branch name note: Recent versions of Git name the default branch `main`; older versions may name it `master`. This guide uses `main` throughout — substitute `master` wherever `main` appears if your system uses it.

> ***Instructor Reference Repository:*** *The public repository at* `https://github.com/ManishGantyala/btech-devops-labs` *shows the expected file structure and commit history for all twelve experiments. Use it as a reference to check your own work at any stage; you may also fork it to keep a personal copy in your GitHub account. A fork of this repository is not your working repository — Experiment 02 specifically teaches you to create a repository from scratch, configure a remote, push, pull, and clone. That repository you create here is what you continue using from Experiment 03 onward.*

Concept

Why Version Control Is Needed

While building software, files are modified constantly. Keeping copies like `script.js`, `script_v2.js`, `script_final.js` by hand quickly becomes unmanageable — and gets worse the moment more than one person is editing the same project.

A **Version Control System (VCS)** solves this: it automatically records every change made to a project, so any earlier version can be recovered, changes can be compared, and multiple people can work on the same codebase without overwriting each other.

Git is the version control system used in this experiment. **GitHub** is where a Git project can be hosted online so it can be shared.

What Is Git?

Git is a **distributed version control system** that runs on your own computer and tracks the history of a project.

Distributed means every developer's computer holds a **complete copy** of the project's history — not just the latest files, but every past version too. This is why most Git commands work instantly with no internet connection; a network is only needed when sharing changes with someone else.

What Is GitHub?

GitHub is an online platform that hosts Git repositories. Git itself has no built-in "cloud" — GitHub is what turns a project sitting on one computer into something that can be shared, backed up, and worked on by a team.

```
Your Computer                GitHub
------------------           ------------------
Local Git Repository  --push-->  GitHub Repository
                      <--pull--
```

Git vs GitHub

	Git	GitHub
What it is	A version control system	A website that hosts Git repositories
Where it runs	Installed on your computer	Accessed online
Works offline?	Yes, for local operations	No, requires a network connection
What it gives you	Commands to track and manage changes	A place to share, back up, and collaborate on repositories

Core Vocabulary

Term	Meaning
Repository (repo)	The location where Git stores a project's files and their full history. Can be *local* (your computer) or *remote* (e.g., on GitHub).
Working directory	The actual project folder on disk, where you create and edit files.
Staging area	A holding area where you choose exactly which changes should go into the *next* commit.
Commit	A saved checkpoint of staged changes, with a message describing what changed.
Branch	An independent line of development, so a feature can be built without disturbing the main codebase.
Remote	A copy of the repository stored somewhere else — in this experiment, on GitHub. `origin` is the conventional name given to a project's main remote.

Basic Git Workflow

```
Working Directory --(git add)--> Staging Area --(git commit)--> Local Repository
                                                                       |
                                                                  (git push)
                                                                       v
                                                                GitHub Repository
                                                                       |
                                                                  (git pull)
                                                                       v
                                                                back into Local Repository
```

Procedure — Part 1: Git Basics (Practice Repository)

Steps 1–13 use a small practice folder called `git-demo` to learn individual commands in isolation. This folder is disposable — its only purpose is to get comfortable with Git commands before you set up your actual working repository in Part 2.

Step 1 — Check Git Installation

```
git --version
```

Observe: A version number is displayed in the form:

```
git version 2.x.x
```

Step 2 — Configure Git with Your Name and Email

Every commit permanently records who made it. This only needs to be done once per computer.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
```

Use your real name and the email address you used for your GitHub account — this is how GitHub links your commits to your profile.

Observe: `user.name` and `user.email` appear in the `--list` output with the values you set.

Step 3 — Create a Practice Directory

```
mkdir git-demo
cd git-demo
```

This folder is now your working directory for Steps 3–13.

Step 4 — Initialize a Repository

`git init` turns an ordinary folder into a Git repository by creating a hidden `.git` folder inside it, where Git stores all history and tracking information.

```
git init
```

Observe: Git reports the new repository was initialized. Run `ls -a` to confirm the hidden `.git` folder exists inside `git-demo`.

Step 5 — Check Repository Status

`git status` reports the current state of the repository — which branch you are on and which files are untracked, modified, or staged.

```
git status
```

Observe: With an empty new repository, Git reports the current branch name and that there is nothing to commit yet. This is the expected starting state.

Step 6 — Create a File

```
echo "Git and GitHub Experiment" > README.txt
git status
```

Observe: `README.txt` is listed as an **untracked file** — Git can see it exists but has not been told to track it. Every new file starts in this state.

Step 7 — Stage the File

`git add` moves a change from the working directory into the staging area, choosing what will go into the next commit.

```
git add README.txt
git status
```

Observe: `README.txt` now appears under "Changes to be committed." It has moved from untracked to staged.

To stage multiple files, or everything in the current directory at once:

```
git add file1.txt file2.txt
git add .
```

Caution: `git add .` stages *every* changed and untracked file in the current directory tree — including files you may not intend to commit, such as configuration files containing passwords. Run `git status` first to review what will be staged.

Step 8 — Commit the Staged Changes

`git commit` permanently records the staged changes as a checkpoint in the repository's history.

```
git commit -m "Add README file"
```

The `-m` message should describe *what changed* — this is what makes history readable later:

Style	Example
Good	`Add event registration form`
Not useful	`changes`

Observe: `git status` now reports a clean working tree with nothing left to commit. The commit is permanently saved.

Step 9 — View Commit History

```
git log
git log --oneline
```

`git log` shows full commit details (author, date, message, commit hash). `--oneline` compresses each commit to a single line — useful once a project has many commits.

Observe: The commit made in Step 8 appears, most recent first.

Step 10 — View Changes with `git diff`

```
git diff           # shows changes in the working directory not yet staged
git diff --staged  # shows changes that are staged and about to be committed
```

Run `git diff` before staging to review what you are about to add. Run `git diff --staged` before committing to review exactly what the commit will contain.

Observe: Immediately after Step 8, both commands produce no output — this is correct, since there is nothing modified or staged beyond what was just committed.

Procedure — Part 1 continued: Branches

A branch is a separate line of development so that a feature can be built without changing `main` until it is ready.

```
main
 |
 +---- feature-login
```

Step 11 — View Branches

```
git branch
```

Observe: The current branch is marked with `*`, for example `* main`.

Step 12 — Create and Switch to a Branch

```
git branch feature-login     # creates the branch (does not switch to it)
git switch feature-login     # switches to it
```

Both actions can be combined into one command:

```
git switch -c feature-login
```

Observe: `git branch` now shows `* feature-login` — the asterisk marks the active branch.

Step 13 — Commit Something on the Branch, Then Merge

Make a small change on the branch so there is something to merge:

```
echo "Feature work" >> README.txt
git add README.txt
git commit -m "Add feature note on feature-login branch"
```

Switch back to `main` and merge the branch:

```
git switch main
git merge feature-login
```

```
feature-login --(git merge)--> main
```

Observe: `git log --oneline` on `main` now shows the commit made on `feature-login` — it has been brought into `main`.

Procedure — Part 2: Your Working Repository (Used from Experiment 03 Onwards)

The `git-demo` folder was for practice only. Part 2 creates the repository you will use for all remaining experiments — it contains your Experiment 01 application files and will be pushed to your own GitHub account. Experiment 03 continues directly from this repository.

Step 14 — Create Your GitHub Repository

1. Open `https://github.com` in your browser and sign in to your account.
2. Click the **+** icon at the top-right of any GitHub page and select **New repository**.
3. In the **Repository name** field, enter a name — for example, `devops-lab`. This is your personal repository; choose any name you like.
4. Set the visibility to **Public** or **Private** — either works for this series.

5. Leave **Add a README file**, **Add .gitignore**, and **Choose a license** all **un-ticked**. The repository must start completely empty so that pushing from your local machine works without conflicts.
6. Click **Create repository**.
7. On the blank repository page that appears, copy the HTTPS URL — it looks like `https://github.com/<your-username>/devops-lab.git`. You will need this URL in Step 17.

Confirm: The repository page shows "This repository is empty" — this is correct.

Step 15 — Create the Local Working Directory

Move out of the `git-demo` folder and create a new directory for your working repository:

```
cd ..
mkdir devops-lab
cd devops-lab
git init
```

Observe: `git init` reports a new repository was initialized inside `devops-lab`. Run `ls -a` to confirm the `.git` folder is present.

Step 16 — Add Your Experiment 01 Files

The Experiment 01 application files go inside an `experiment-01/` subfolder. This folder structure (`experiment-01/`, `experiment-02/`, …) mirrors how the experiments are organized throughout the series.

```
mkdir experiment-01
```

Copy your three Experiment 01 files into `experiment-01/`. Adjust the source path to wherever your files are saved:

```
cp /path/to/your/index.html experiment-01/
cp /path/to/your/style.css  experiment-01/
cp /path/to/your/script.js  experiment-01/
```

Confirm the files are in place:

```
ls experiment-01/
```

Observe: `index.html`, `style.css`, and `script.js` are listed inside `experiment-01/`.

Step 17 — Stage and Commit the Experiment 01 Files

```
git add experiment-01/
git commit -m "Add Experiment 01 event registration application"
```

Check the result:

```
git status
git log --oneline
```

Observe: `git status` reports a clean working tree. `git log --oneline` shows the commit you just made.

Step 18 — Connect the Local Repository to Your GitHub Repository

Register your GitHub repository as the remote named `origin`:

```
git remote add origin <your-repository-url>
```

Replace `<your-repository-url>` with the HTTPS URL copied in Step 14.

Verify the connection was registered:

```
git remote -v
```

Observe:

```
origin  https://github.com/<your-username>/devops-lab.git (fetch)
origin  https://github.com/<your-username>/devops-lab.git (push)
```

`(fetch)` is the URL Git uses when *downloading* from GitHub. `(push)` is the URL Git uses when *uploading* to GitHub. Both should show your own repository URL.

Step 19 — Push to GitHub

```
git push -u origin main
```

The `-u` flag links your local `main` branch to the remote `main` branch, so future pushes in this repository only require `git push`.

Observe: After the push completes, open your GitHub repository in a browser. The `experiment-01/` folder and the three files inside it should be visible. This is your working repository — it is what Experiment 03 continues from.

Step 20 — Pull from GitHub

`git pull` downloads any new commits from the remote and merges them into your local branch. In a solo workflow you may not have anything new to pull, but the command is essential when collaborators push changes or when you merge a Pull Request on GitHub (as you will do in Experiment 03).

```
git pull origin main
```

Observe: Because you just pushed, Git reports "Already up to date." — this is the correct, expected result. There is no error.

Step 21 — Clone a Repository

`git clone` creates a brand-new local copy of a repository from scratch. Use it when a repository exists on GitHub but not yet on your computer. To demonstrate, clone your own repository into a separate folder:

```
cd ..
git clone <your-repository-url> devops-lab-clone
cd devops-lab-clone
git status
ls experiment-01/
```

Observe: `git status` shows a clean working tree. The `experiment-01/` folder and all three files are present — cloning downloads everything automatically. Unlike `git init`, `git clone` also sets up `origin` automatically: run `git remote -v` to confirm it already points to your GitHub repository.

Return to your main working directory:

```
cd ../devops-lab
```

Push, Pull, and Clone — Summary

Command	Direction	When to use it
`git push`	Local → GitHub	After committing, to upload your commits to GitHub
`git pull`	GitHub → Local	When GitHub has new commits your local copy does not have yet
`git clone`	GitHub → New local folder	When you need a fresh local copy of a repository that does not exist on your machine yet

Verification

Check	What to look for
`git status` (in `devops-lab/`)	Working tree is clean; nothing left uncommitted
`git log --oneline` (in `devops-lab/`)	At least one commit with a meaningful message

Check	What to look for
`git branch`	`main` branch exists
`git remote -v`	`origin` points to your own GitHub repository URL
GitHub repository page in browser	`experiment-01/` folder with `index.html`, `style.css`, `script.js` is visible
`git status` (in `devops-lab-clone/`)	Clean working tree; `experiment-01/` files present

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
`fatal: not a git repository` error	Command ran outside a Git-tracked folder	`pwd` to confirm location; `ls -a` to check for `.git`; `cd` into the repo folder	Re-run the Git command
A changed file is missing from the commit	`git add` was skipped — unstaged files are never included in commits	`git status` shows the file in red (not staged); `git add <file>` to stage it	`git commit -m "..."`
Changes are staged (green) but `git log` shows nothing new	`git commit` was never run after `git add`	`git status` shows "Changes to be committed" — run `git commit -m "..."`	`git log --oneline` to confirm the commit
Commits appear locally but not on GitHub	`git push` was not run after committing	`git push`	Refresh the GitHub repository page
Your edit appeared on the wrong branch	You edited without switching to the intended branch first	`git branch` to see which branch is active (marked `*`); switch with `git switch <branch>` before editing	Check `git branch` before each future edit
`git push` fails: "updates were rejected" on first push	GitHub repo was created with a README or license file, so remote and local histories diverge	Recreate the repo with all options un-ticked, or run `git pull origin main --allow-unrelated-histories` once	`git push -u origin main`
`git push` fails: "403" or		`git remote -v` — the URL must contain your	`git push`

Problem	Why it happens	What to check/do	Retry / Next
"Permission denied"	The remote URL points to someone else's repository	own GitHub username; update with `git remote set-url origin <your-url>`	

Quick Reference

Command	Purpose
`git --version`	Check installed Git version
`git config --global user.name` / `user.email`	Set your identity for commits
`git init`	Initialize a new local repository
`git status`	Check current repository state
`git add`	Stage changes for the next commit
`git commit -m`	Save staged changes as a commit
`git log` / `git log --oneline`	View commit history
`git diff` / `git diff --staged`	View unstaged / staged changes
`git branch`	List or create branches
`git switch`	Change the current branch
`git switch -c`	Create and switch to a new branch
`git merge`	Combine another branch into the current one
`git remote add origin`	Register a remote repository
`git remote -v`	View registered remotes
`git push -u origin main`	Push and link the branch to the remote (first push)
`git push`	Push after the first time
`git pull`	Download and merge remote changes
`git clone`	Create a local copy of a remote repository

Result

This experiment covered the essential Git and GitHub commands: configuring Git, initializing a local repository, tracking files through the staging area into commits,

viewing history and diffs, creating and merging branches, and connecting a local repository to a personal GitHub repository for pushing, pulling, and cloning. By the end of Part 2, a working repository containing the Experiment 01 application files is live on your own GitHub account — this repository is what Experiment 03 continues from.

Experiment 03 — Practice Source Code Management on GitHub

Aim

To practice source code management on GitHub by taking the real application built in Experiment 01 (the TechFest 2026 event registration form) through a feature branch → Pull Request → review → merge workflow.

Learning Objectives

- Explain what a feature branch is for and why changes are made on one instead of directly on `main`.
- Create a branch, commit a change to real project source code, and push that branch to GitHub.
- Open a Pull Request on GitHub and review its diff before merging.
- Merge a Pull Request and bring the merged change back into the local repository.
- Verify, both locally and on GitHub, that a change was managed correctly from branch to merge.

This experiment assumes Git and GitHub basics from Experiment 02 are already understood, and that the working repository created in Experiment 02 is available.

Requirements

- Your own GitHub repository created in Experiment 02 (the `devops-lab` repository, or whatever name you chose), with the Experiment 01 source code (`experiment-01/index.html`, `style.css`, `script.js`) already committed and pushed to GitHub.
- Git installed, and Git configured with your user name and email (Experiment 02, Step 2).
- A GitHub account with push access to your own GitHub repository.
- A terminal or command prompt. Run all commands from inside your `devops-lab/` working directory.

Concept

Experiment 02 covered individual Git and GitHub commands — first with a disposable practice folder, then by pushing the Experiment 01 application to your own GitHub repository. This experiment applies those same commands to the **real application already in your repository**, in the sequence a change is normally managed on GitHub:

```
main --(branch)--> feature branch --(edit, commit)--> pushed branch --(Pull Request)--> reviewed
diff --(merge)--> main

              |

       (pull locally)
```

Two ideas are central here:

- **Feature branch** — a change is developed on its own branch, not directly on `main`, so `main` always stays in a working state.
- **Pull Request (PR)** — GitHub's mechanism for proposing a branch's changes to be merged. A PR shows exactly which lines changed (the diff) and requires a deliberate merge action. Reviewing that diff before merging, even alone, is the core habit this experiment teaches.

Procedure

Step 1 — Confirm the Starting State

Open a terminal and navigate to your `devops-lab/` working directory (the repository you created in Experiment 02). Check that the local repository is on `main` and up to date with GitHub before starting any new work. Starting a feature branch from an outdated or dirty `main` can carry over unrelated changes or conflicts.

```
git switch main
git status
git pull origin main
```

Observe: `git status` reports a clean working tree. `git pull` reports either new commits fetched or "Already up to date." Confirm that `experiment-01/index.html` is present with `ls experiment-01/`.

Step 2 — Create a Feature Branch

Create a new branch off `main` to hold the upcoming change. This keeps `main` — the working registration form — untouched until the change has been reviewed and merged.

```
git switch -c update-registration-form
```

Observe: `git branch` lists `update-registration-form` with a `*` marking it as the current branch.

Step 3 — Make a Small Change to the Experiment 01 Source Code

Open `experiment-01/index.html` and find the submit button line:

```
<button type="submit">Register</button>
```

Change it to:

```
<button type="submit">Register Now</button>
```

Save the file.

```
git status
git diff
```

Observe: `git status` lists `experiment-01/index.html` as modified. `git diff` shows only the single changed line — the button text — and nothing else.

Step 4 — Stage and Commit the Change

```
git add experiment-01/index.html
git commit -m "Update submit button label to Register Now"
```

Observe: `git status` reports a clean working tree; `git log --oneline` shows the new commit on top.

Step 5 — Push the Feature Branch to GitHub

A branch must exist on GitHub before a Pull Request can be opened from it.

```
git push -u origin update-registration-form
```

Observe: GitHub's push output includes a link to open a Pull Request for the branch. The branch also becomes visible in the repository's branch dropdown on GitHub.

Step 6 — Open a Pull Request

1. Go to the repository on GitHub.
2. Click **Compare & pull request** for the pushed branch (or **New pull request**, then select the branch).
3. Confirm the base branch is `main` and the compare branch is `update-registration-form`.
4. Add a short title and description.
5. Click **Create pull request**.

Observe: The Pull Request page opens, showing its status as open and unmerged.

Step 7 — Review the Diff

Open the **Files changed** tab on the Pull Request.

Observe: Only the intended file(s) are listed, and the highlighted additions/removals match the edit made in Step 3 — nothing unrelated is included. This review step distinguishes managed source code from an unreviewed `git push`.

Step 8 — Merge the Pull Request

Click **Merge pull request**, then **Confirm merge**.

Observe: The Pull Request status changes to **Merged**. The repository's default branch view now shows the change in `main` on GitHub.

Step 9 — Pull the Merged Change Back Locally

The merge happened on GitHub, not on the local machine — the local `main` is still behind until it is pulled.

```
git switch main
git pull origin main
```

Observe: `git log --oneline` shows the merged commit on `main`.

Step 10 — Clean Up the Feature Branch

```
git branch -d update-registration-form
git push origin --delete update-registration-form
```

Observe: `git branch` no longer lists the feature branch locally, and it no longer appears in the branch dropdown on GitHub.

Verification

Check	Where	Confirms
`git branch`	Local	Feature branch was created, then removed after merge
`git log --oneline` on `main`	Local	Merged commit is present after `git pull`
Pull Request **Files changed** tab	GitHub	Diff matched the intended change before merging
Pull Request status	GitHub	Shows **Merged**, not left open
Repository file view	GitHub	`experiment-01` file reflects the merged change
`git status`	Local	Working tree is clean after the full cycle

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
Changes are on `main` but needed on a feature branch	You edited without creating or switching to a branch first	`git branch` to check which branch is active; always run `git switch -c <branch>` before editing	Move uncommitted changes: `git stash`, switch to the new branch, then `git stash pop`
GitHub shows no branch to compare when opening a PR	The branch was never pushed to GitHub	`git push -u origin update-registration-form`	Return to GitHub — the "Compare & pull request" button will appear
PR was merged without reviewing the diff	**Files changed** tab was skipped before clicking Merge	On any open PR, open **Files changed** first and confirm only intended lines are highlighted	For already-merged unwanted changes, open a new PR to revert them
Local `main` doesn't show the merged commit	The merge happened on GitHub; `git pull` was not run locally	`git switch main && git pull origin main`	`git log --oneline` — the merged commit should now appear
Old merged branches accumulate in the branch list	Branches are not deleted after the PR is closed	`git branch -d <branch>` locally; `git push origin --delete <branch>` on GitHub	`git branch` and the GitHub branch list should no longer show the old branch

Result

The Experiment 01 event registration source code was carried through a complete source code management cycle on GitHub: a feature branch was created, a change was made and committed, the branch was pushed, a Pull Request was opened and its diff reviewed, the Pull Request was merged into `main`, and the merged change was pulled back into the local repository.

Experiment 04 — Jenkins Installation and Setup

Aim

To install Jenkins, start it as a running service, complete its initial setup, and reach a working Jenkins dashboard with an admin account.

Learning Objectives

- Explain what Jenkins is and why it needs a specific Java version to run.
- Add the Jenkins package repository and install Jenkins via the package manager.
- Start, enable, and check the status of the Jenkins service.
- Unlock Jenkins using its initial admin password and complete the setup wizard.
- Reach and confirm a working Jenkins dashboard.

This experiment ends the moment the Jenkins dashboard is reached with an admin account created. Creating jobs, pipelines, or Jenkinsfiles, and connecting Jenkins to GitHub belong to Experiment 05.

Requirements

- A Linux system (Ubuntu/Debian-based) with `sudo` access and internet connectivity.
- A web browser, to complete the setup wizard.

Concept

Jenkins is an open-source **automation server**, most commonly used to automatically build, test, and deploy software whenever code changes. This overall practice is called CI/CD (Continuous Integration / Continuous Deployment). This experiment only gets Jenkins installed and reachable so that Experiment 05 has something to build on.

A few facts about Jenkins shape every step below:

- **Jenkins is a Java application.** A compatible Java Development Kit (JDK) must exist on the machine before Jenkins can start.
- **Jenkins is operated through a web interface**, by default on port `8080`. Almost everything — including finishing the install itself — happens in a browser.
- **Jenkins runs as a background service**, managed via `systemctl`, so it keeps running independently of any terminal session.
- **The first time Jenkins starts, it locks itself** and writes a one-time random password to a local file. This proves whoever is completing the setup has file access on the server, not just network access to port 8080.

- **Plugins are how Jenkins gains functionality.** The setup wizard's "install suggested plugins" step installs the standard baseline set.

```
Java installed --> Jenkins installed --> Jenkins service running --> Unlock Jenkins (browser)
                                                                              |
                                                                     Install plugins
                                                                              |
                                                                     Create admin user
                                                                              |
                                                                      Jenkins dashboard
```

Procedure

Step 1 — Check/Install the Java Prerequisite

Jenkins is a Java application and will refuse to run without a supported JDK.

```
java -version
```

If Java is missing or unsupported, install a supported LTS JDK (Jenkins currently supports Java 17 and Java 21):

```
sudo apt update
sudo apt install fontconfig openjdk-17-jre
```

Observe: `java -version` reports an installed version in the range Jenkins supports.

Step 2 — Add the Jenkins Package Repository

Linux package managers work from a list of known repositories — servers that host installable software packages. The OS ships with a default list, but Jenkins is not on it. Adding Jenkins's own repository tells `apt` where to look for the `jenkins` package.

A **signing key** is a cryptographic credential used to verify that each package actually came from the legitimate Jenkins project and was not tampered with in transit.

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key

echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
```

Observe: `apt update` completes without errors referencing the Jenkins repository, and `jenkins` becomes available as an installable package.

Step 3 — Install Jenkins

```
sudo apt install jenkins
```

Observe: The install completes without errors, and the Jenkins service is created.

Step 4 — Start and Enable the Jenkins Service

Start the Jenkins service now, and enable it to start automatically on future boots.

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Observe: Both commands complete without error output.

Step 5 — Check the Jenkins Service Status

Confirm Jenkins is actually running before trying to open it in a browser. Checking service status first avoids confusing a "service failed to start" problem with a "wrong URL/port" problem later.

```
sudo systemctl status jenkins
```

Observe: The status reports `active (running)`. If it shows `failed` or `inactive`, this must be resolved before continuing.

Step 6 — Open Jenkins on Port 8080

From this point on, setup is completed through the web UI.

Open a browser and navigate to:

```
http://localhost:8080
```

Observe: An **"Unlock Jenkins"** page loads, asking for an administrator password.

Step 7 — Retrieve the Initial Admin Password

This password proves the setup is being completed by someone with access to the server's filesystem.

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy the printed value and paste it into the "Unlock Jenkins" page.

Observe: Submitting the password moves the wizard forward to the plugin selection screen.

Step 8 — Install Suggested Plugins

Click **"Install suggested plugins"** and wait for the installation progress screen to finish.

Observe: A progress screen lists each plugin as it installs, ending with all items marked complete.

Step 9 — Create the First Admin User

Fill in the requested username, password, full name, and email address on the "Create First Admin User" screen, then continue.

Observe: The wizard proceeds to the instance configuration screen without validation errors.

Step 10 — Confirm the Jenkins URL

Leave the pre-filled URL as-is (typically `http://localhost:8080/`) and click **Save and Finish**.

Observe: A "Jenkins is ready!" confirmation screen appears.

Step 11 — Reach the Jenkins Dashboard

Click **Start using Jenkins**.

Observe: The Jenkins dashboard loads, showing an empty job list and the left-hand navigation menu (New Item, Manage Jenkins, etc.).

Verification

Check	Where	Confirms
`java -version`	Terminal	A Jenkins-supported JDK is installed
`systemctl status jenkins`	Terminal	Jenkins service is `active (running)`
`http://localhost:8080` loads	Browser	Jenkins web server is reachable
Initial admin password accepted	Terminal + Browser	Setup wizard can be unlocked
Suggested plugins finish installing	Browser	Baseline Jenkins functionality is present
Login with created admin user succeeds	Browser	Admin account was created correctly
Jenkins dashboard loads	Browser	Installation and setup completed successfully

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
Jenkins service shows `failed` in `systemctl status`	An unsupported Java version is installed — Jenkins requires Java 17 or 21	`java -version`; if it shows Java 11 or earlier, run `sudo apt install openjdk-17-jre`	`sudo systemctl restart jenkins`; confirm status shows `active (running)`
`apt install jenkins` reports "package not found"	`apt update` was not run after adding the Jenkins repository	`sudo apt update` after Step 2	`sudo apt install jenkins`
Browser shows "connection refused" on `http://localhost:8080`	Jenkins is not running yet, or port 8080 is in use by another process	`sudo systemctl status jenkins` first; if running but still failing, `sudo lsof -i :8080` to check for a port conflict	Open the browser only once Jenkins shows `active (running)`
`cat initialAdminPassword` gives "No such file or directory"	Wrong path, or `sudo` was omitted	`sudo cat /var/lib/jenkins/secrets/initialAdminPassword` — exact path, with `sudo`	Paste the printed value into the Unlock Jenkins page
Port 8080 already in use — Jenkins starts then immediately fails	Another process claimed port 8080 before Jenkins	`sudo lsof -i :8080` to identify the process; stop it	`sudo systemctl restart jenkins`
One or two plugins fail during "Install suggested plugins"	Transient network issue during plugin download	Continue the wizard — individual plugin failures at this step are not fatal	Retry failed plugins from **Manage Jenkins → Plugins**

Result

Jenkins was installed and configured up to a working state: the Java prerequisite was satisfied, the Jenkins package repository was added, Jenkins was installed and started as a service, the initial admin password unlocked the setup wizard, the suggested plugins were installed, the first admin user was created, and the Jenkins dashboard was successfully reached.

Experiment 05 — Demonstrate CI/CD Using Jenkins

Aim

To demonstrate Continuous Integration and Continuous Deployment (CI/CD) using a Jenkins Pipeline connected to this project's GitHub repository — first run manually, then extended to trigger automatically on a push to `main`.

Learning Objectives

- Explain why a GitHub Personal Access Token (PAT) is needed for Jenkins to access a GitHub repository over HTTPS.
- Create a Jenkins Pipeline job connected to a GitHub repository using the Jenkins UI.
- Define pipeline stages that check out, validate, deploy, and verify an application.
- Trigger a pipeline manually and confirm successful execution from its console output.
- Explain why a locally running Jenkins instance (inside WSL2) cannot directly receive a GitHub webhook, and how ngrok addresses that.
- Configure a GitHub webhook so a push to `main` automatically triggers the Jenkins pipeline, and distinguish a webhook-triggered build from a manually triggered one using its console output.

This experiment assumes Jenkins is already installed, running, and reachable at its dashboard — that is Experiment 04.

Requirements

- A running Jenkins instance (Experiment 04), running locally inside WSL2.
- This project's GitHub repository, with a GitHub Personal Access Token (PAT) for Jenkins to authenticate against it.
- Nginx installed on the Jenkins host, to serve the deployed application.
- Write access to the deployment target directory, `/var/www/jenkins-demo`.
- For Part B only: ngrok, to expose the local Jenkins instance to the internet for GitHub webhook delivery.

Concept

A Jenkins **Pipeline** job defines an ordered sequence of stages that Jenkins runs every time the job executes. In this experiment, the pipeline's stages are:

```
Checkout --> Validate --> Deploy --> Verify
```

- **Checkout** — pulls the latest source from the GitHub repository.
- **Validate** — checks the pulled source before it is used further.
- **Deploy** — publishes the application to `/var/www/jenkins-demo`, served by Nginx.
- **Verify** — confirms the deployment succeeded by checking the deployed files exist.

This experiment is carried out in two parts, in the order they were actually done:

- **Part A** — the pipeline started manually (**Build Now**). This is the original working implementation and proves the CI/CD stages themselves work correctly.
- **Part B** — the same pipeline extended so a `git push` to `main` on GitHub starts it automatically, with no manual click. Part B was added *after* Part A was already confirmed working. Do not attempt Part B until Part A produces `Finished: SUCCESS`.

Prerequisites — Before Part A

Run these commands on the Jenkins host (your WSL2 terminal) before creating any Jenkins job.

1. Install and start Nginx:

```
sudo apt update
sudo apt install nginx
sudo systemctl start nginx
sudo systemctl enable nginx
```

2. Create the deployment directory and grant Jenkins write access:

```
sudo mkdir -p /var/www/jenkins-demo
sudo chown -R jenkins:jenkins /var/www/jenkins-demo
```

`/var/www/jenkins-demo` is where the Deploy stage copies the application files. The `jenkins` OS user runs the pipeline — if it does not own this directory, the Deploy stage will fail with a permission error.

Part A — Manual Pipeline Run

(Original, manually-triggered implementation. Complete all six steps before moving to Part B.)

Step 1 — Create a GitHub Personal Access Token and Add It to Jenkins

Why: GitHub no longer accepts an account password for Git over HTTPS. Jenkins needs a Personal Access Token (PAT) in its place.

Create the PAT on GitHub:

1. Sign in to GitHub. Click your avatar (top-right) and choose **Settings**.
2. Scroll to the bottom of the left sidebar and click **Developer settings**.
3. Click **Personal access tokens → Tokens (classic)**.
4. Click **Generate new token (classic)**.
5. Fill in **Note** (e.g., `Jenkins CI`) and choose an expiration.
6. Under **Select scopes**, tick **repo** (for a public repository, `public_repo` is sufficient).
7. Click **Generate token**. Copy the token value immediately — GitHub will not show it again.

Add the PAT to Jenkins:

1. Open `http://localhost:8080` in your browser. This is the Jenkins dashboard.
2. In the left sidebar, click **Manage Jenkins**.
3. Click **Credentials**.
4. Under **Stores scoped to Jenkins**, click **System**, then click **Global credentials (unrestricted)**.
5. Click **Add Credentials** in the left sidebar.
6. Set **Kind** to `Username with password`.
7. In **Username**, enter your GitHub username.
8. In **Password**, paste the PAT you copied from GitHub.
9. In **ID**, type exactly: `github-btech-devops` (this ID is referenced by name in the Jenkinsfile — it must match).
10. Click **Create**.

Confirm: The credential `github-btech-devops` now appears in the Global credentials list.

Step 2 — Create the Jenkins Pipeline Job

Why: A Pipeline job type (as opposed to a Freestyle job) is required because it reads a `Jenkinsfile` from the repository — that file defines the ordered Checkout, Validate, Deploy, Verify stages.

Create the job:

1. On the Jenkins dashboard (`http://localhost:8080`), click **New Item** in the left sidebar.
2. In the **Enter an item name** field, type a name for this job — for example, `devops-lab-pipeline`.
3. Click **Pipeline** in the list of job types below the name field.
4. Click **OK** at the bottom. Jenkins opens the job's configuration page.

Configure the Pipeline source:

5. Scroll down to the **Pipeline** section (near the bottom of the configuration page).

6. In the **Definition** drop-down, change `Pipeline script` to **Pipeline script from SCM**. New fields appear.
7. In the **SCM** drop-down, select **Git**.
8. In **Repository URL**, enter your GitHub repository URL. For this experiment the URL is: `https://github.com/ManishGantyala/btech-devops-labs.git`
9. In **Credentials**, click the drop-down and select **github-btech-devops** — the credential created in Step 1. If the URL and credential are correct, any red error text under the URL field disappears.
10. Under **Branches to build**, find the **Branch Specifier** field. Change `*/master` to `*/main`.
11. In **Script Path**, the default value is `Jenkinsfile`. Change it to `experiment-05/Jenkinsfile` — this is the path to the pipeline script inside the repository.
12. Click **Save** at the bottom. Jenkins returns you to the pipeline job's main page.

Confirm: The job's main page shows the name you chose, and the left sidebar contains **Build Now** and **Configure** links.

Step 3 — The Jenkinsfile (Pipeline Script)

The pipeline script is `experiment-05/Jenkinsfile` in this repository. Jenkins reads this file automatically each time the job runs — you do not type it into Jenkins manually. The file contains:

```
pipeline {
    agent any

    stages {

        stage('Checkout') {
            steps {
                checkout([
                    $class: 'GitSCM',
                    branches: [[name: '*/main']],
                    userRemoteConfigs: [[
                        credentialsId: 'github-btech-devops',
                        url: 'https://github.com/ManishGantyala/btech-devops-labs.git' //Replace
with your GitHub repository URL
                    ]]
                ])
            }
        }

        stage('Validate') {
            steps {
                sh '''
                    test -f experiment-01/index.html
                    test -f experiment-01/style.css
                    test -f experiment-01/script.js
                    echo "Experiment 01 files validated successfully."
                '''
```

```
            }
        }

        stage('Deploy') {
            steps {
                sh '''
                    rm -rf /var/www/jenkins-demo/*
                    cp -r experiment-01/* /var/www/jenkins-demo/
                    echo "Application deployed successfully."
                '''
            }
        }

        stage('Verify') {
            steps {
                sh '''
                    test -f /var/www/jenkins-demo/index.html
                    test -f /var/www/jenkins-demo/style.css
                    test -f /var/www/jenkins-demo/script.js
                    echo "Deployment verification successful."
                '''
            }
        }
    }
}
```

Note that `credentialsId: 'github-btech-devops'` in the Checkout stage references the credential ID entered in Step 1. They must match exactly, or the Checkout stage will fail.

Step 4 — Trigger the Pipeline Manually (Build Now)

1. From the pipeline job's main page, click **Build Now** in the left sidebar.
2. Look at the **Build History** panel in the lower-left. A new row appears, showing a build number (e.g., `#1`) with a progress icon. If the icon is animated, the build is running. Wait for it to settle — a blue circle means success; a red circle means failure.
3. Click on the build number (e.g., **#1**) to open that build's detail page.
4. On the build detail page, click **Console Output** in the left sidebar. The full log of everything Jenkins did appears here.

Step 5 — Read the Console Output

The Console Output shows every action Jenkins took, stage by stage. Read through it top to bottom and look for the following:

Checkout stage — Jenkins clones or updates the repository from GitHub. Expect lines such as:

```
Cloning the remote Git repository
Checking out Revision ...
```

If the Checkout stage fails, the most common cause is a credential mismatch — confirm the credential ID in the Jenkinsfile matches what was created in Step 1.

Validate stage — Jenkins checks that the three required files exist in the workspace:

```
Experiment 01 files validated successfully.
```

If this fails, the `experiment-01/index.html`, `experiment-01/style.css`, or `experiment-01/script.js` files were not found in the checked-out repository.

Deploy stage — Jenkins copies the application files to `/var/www/jenkins-demo/`:

```
Application deployed successfully.
```

If this stage fails with a permission error, the Jenkins OS user does not have write access to `/var/www/jenkins-demo/` — re-run the `chown` command from the Prerequisites section.

Verify stage — Jenkins confirms the three files now exist at the deployment location:

```
Deployment verification successful.
```

Final line — the last line of a successful run is:

```
Finished: SUCCESS
```

If this line is not present, scroll up through the Console Output to find the first error — it will be in the stage that failed.

Step 6 — Confirm the Deployed Files

Part A is complete once `Finished: SUCCESS` appears. To also confirm the deployment at the OS level, run this in a WSL2 terminal on the Jenkins host:

```
ls /var/www/jenkins-demo/
```

The output should show `index.html`, `style.css`, and `script.js` — the files that were copied from `experiment-01/` by the Deploy stage.

Next: Part A is complete. Begin Part B only after the Console Output shows `Finished: SUCCESS` — Part B extends the same working pipeline with automatic triggering and requires Part A to already be confirmed.

Part B — Automatic Trigger on GitHub Push

(Added after Part A was confirmed working. Do not start Part B until Part A produces `Finished: SUCCESS`.)

Step 1 — Why Part B Needs ngrok

Part A proved the pipeline works. The goal of Part B is to make the same pipeline start automatically whenever a commit is pushed to `main` on GitHub, without a manual **Build Now** click.

GitHub does this by sending an HTTP request — called a **webhook** — to the Jenkins server each time a push happens. For that to work, GitHub must be able to reach Jenkins over the internet.

Jenkins is running locally inside WSL2, which has no public IP address and is not reachable from the internet. This is the specific obstacle Part B must solve. **ngrok** solves it by creating a secure public tunnel: it gives you a temporary public URL (`https://<random>.ngrok-free.app`) that forwards incoming requests to `http://localhost:8080` on your machine. GitHub sends the webhook to that public URL; ngrok forwards it to Jenkins.

Step 2 — Install ngrok

Run these commands once to install ngrok on the Jenkins host (WSL2):

```
curl -s https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee /etc/apt/trusted.gpg.d/
ngrok.asc >/dev/null
echo "deb https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee /etc/apt/sources.list.d/
ngrok.list
sudo apt update && sudo apt install ngrok
```

Sign up for a free account at `https://ngrok.com`. After signing up, copy your auth token from the ngrok dashboard and run:

```
ngrok config add-authtoken <your-token>
```

Replace `<your-token>` with the actual token from your ngrok account.

Step 3 — Start the ngrok Tunnel

Open a **new, separate terminal window** and run:

```
ngrok http 8080
```

Keep this terminal open for the entire duration of Part B — closing it ends the tunnel and breaks webhook delivery.

ngrok displays a status screen. Look for the **Forwarding** line:

```
Forwarding   https://<random-subdomain>.ngrok-free.app -> http://localhost:8080
```

Copy the full `https://...ngrok-free.app` URL — you will paste it into the GitHub webhook configuration in the next step.

Step 4 — Add the Webhook in GitHub

1. Open your GitHub repository in a browser.
2. Click **Settings** (the tab at the top of the repository page).
3. In the left sidebar, click **Webhooks**.
4. Click **Add webhook**.
5. In **Payload URL**, paste the ngrok forwarding URL copied in Step 3, then append `/github-webhook/` at the end, so it looks like: `https://<random-subdomain>.ngrok-free.app/github-webhook/` The trailing slash and the `/github-webhook/` path are required — this is the specific endpoint Jenkins listens on for GitHub events.
6. Set **Content type** to `application/json`.
7. Leave **Which events would you like to trigger this webhook?** set to **Just the push event**.
8. Click **Add webhook**.

Confirm: The webhook now appears in the repository's Webhooks list. GitHub may show a green tick after delivering a test ping to Jenkins.

Step 5 — Enable the GitHub Push Trigger in Jenkins

GitHub's webhook alone is not enough — Jenkins must also be told to react to it.

1. In Jenkins, go to the pipeline job's main page.
2. Click **Configure** in the left sidebar.
3. Scroll to the **Build Triggers** section.
4. Tick the checkbox labelled **GitHub hook trigger for GITScm polling**.
5. Click **Save**.

Confirm: The pipeline job's configuration page shows **GitHub hook trigger for GITScm polling** enabled under Build Triggers.

Step 6 — Push a Commit to `main`

Make any small change to a tracked file and push it:

```
git push origin main
```

Confirm: The new commit appears on GitHub under the repository's commit history for `main`.

Step 7 — Find the Automatically Triggered Build in Jenkins

Within a few seconds of the push, Jenkins should start a new build on its own.

1. Open the pipeline job's main page in Jenkins (`http://localhost:8080/job/<your-job-name>/`).
2. Look at the **Build History** panel (lower-left). A new build number appears — one higher than the last manual build from Part A. This build was started automatically by the webhook; you did not click **Build Now**.
3. Click the new build number to open its detail page.
4. Click **Console Output** in the left sidebar.

Step 8 — Confirm Automatic Trigger and Successful Run

The very first lines of the Console Output identify what started this build. Look for:

```
Started by GitHub push by ManishGantyala
```

This line is the key evidence that distinguishes an automatic, webhook-triggered build from a manually triggered one (which would say `Started by user …` instead).

Then scroll to the end of the Console Output and confirm the pipeline completed:

```
Finished: SUCCESS
```

Both lines must be present for Part B to be considered complete: the first proves the trigger worked; the second proves the pipeline ran successfully.

Verification

Check	Where	Confirms
PAT credential `github-btech-devops` listed in Jenkins	Manage Jenkins → Credentials	Jenkins can authenticate to GitHub
Pipeline job shows repository URL and credential in its configuration	Jenkins → Job → Configure	Job is wired to the correct repository and credential
Script Path set to `experiment-05/Jenkinsfile`	Jenkins → Job → Configure → Pipeline section	Jenkins reads the correct pipeline script
Manual build (Part A) Console Output ends `Finished: SUCCESS`	Console Output for build #1	

Check	Where	Confirms
		The core pipeline — Checkout, Validate, Deploy, Verify — works correctly
`index.html`, `style.css`, `script.js` present at `/var/www/jenkins-demo/`	Jenkins host terminal	Deploy stage actually copied the application files
ngrok terminal shows an active Forwarding URL	ngrok terminal	Local Jenkins is reachable from the internet
Webhook listed under GitHub repository Webhooks settings	GitHub → Settings → Webhooks	GitHub is configured to notify Jenkins on a push to `main`
Jenkins pipeline job shows **GitHub hook trigger for GITScm polling** enabled	Jenkins → Job → Configure → Build Triggers	Jenkins is set to react to incoming webhook calls
New build appears in Jenkins Build History after `git push` without clicking Build Now	Pipeline job page	The webhook actually triggered a build
Automatic build Console Output begins `Started by GitHub push by ManishGantyala`	Console Output	Confirms the build was started by the GitHub webhook, not manually
Automatic build Console Output ends `Finished: SUCCESS`	Console Output	The automatic trigger produces a fully working pipeline run

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
Checkout stage fails: "Authentication failed" or "403"	GitHub no longer accepts account passwords over HTTPS; a PAT is required	In **Manage Jenkins → Credentials**, confirm `github-btech-devops` was created with a PAT (not your GitHub password) in the Password field	**Build Now** — Checkout should succeed
Checkout stage fails: "Invalid credentials" even with a correct PAT	`credentialsId` in the Jenkinsfile doesn't exactly match the credential ID stored in Jenkins	Open the Jenkinsfile — find `credentialsId: 'github-btech-devops'`; confirm it matches the **ID** field in **Manage Jenkins → Credentials** character for character	**Build Now**
Build fails immediately: "Could not	Script Path is still the default `Jenkinsfile`, not	**Job → Configure → Pipeline** — set Script Path	**Build Now**

Problem	Why it happens	What to check/do	Retry / Next
find pipeline script"	`experiment-05/Jenkinsfile`	to `experiment-05/Jenkinsfile`	
Checkout stage fails: "couldn't find remote ref refs/heads/master"	Branch Specifier is still `*/master` but the repository uses `main`	**Job → Configure → Pipeline** — change `*/master` to `*/main`	**Build Now**
Deploy stage fails: "Permission denied" on `/var/www/jenkins-demo/`	The directory is owned by `root`, not the `jenkins` OS user that runs the pipeline	`sudo chown -R jenkins:jenkins /var/www/jenkins-demo`	**Build Now** — Deploy stage should print "Application deployed successfully."
Pushing to `main` on GitHub doesn't start a Jenkins build (Part B)	Either the webhook trigger is not enabled in Jenkins, or the Payload URL is wrong	Confirm **GitHub hook trigger for GITScm polling** is ticked in **Job → Configure → Build Triggers**; confirm the Payload URL in GitHub ends with `/github-webhook/` (trailing slash required)	Push a small commit; a new build should appear in Jenkins within seconds
Webhook was working, now GitHub shows "Failed to deliver" (Part B)	ngrok free-tier assigns a new URL each time it restarts; the old URL no longer forwards to Jenkins	Run `ngrok http 8080` again; copy the new Forwarding URL; update the Payload URL in **GitHub → Settings → Webhooks**	Push a small commit — the build should trigger

Result

CI/CD was demonstrated successfully using a Jenkins Pipeline connected to this project's GitHub repository. In Part A, the pipeline — Checkout, Validate, Deploy, Verify — was configured through the Jenkins UI and run manually via **Build Now**, deploying the `experiment-01` application files to `/var/www/jenkins-demo` behind Nginx, with the console output ending in `Finished: SUCCESS`. In Part B, automatic triggering on a push to `main` was subsequently added and verified using a GitHub webhook tunneled through ngrok (required because Jenkins runs locally inside WSL2); the resulting build's console output began with `Started by GitHub push by ManishGantyala` and again ended in `Finished: SUCCESS`, confirming the automatic trigger produced a fully successful pipeline run.

Experiment 06 — Explore Docker Commands for Content Management

Aim

To explore Docker commands used for content management — obtaining images, creating and running containers, viewing and controlling container state, accessing and copying content, and removing containers and images.

Learning Objectives

- Explain why Docker exists and what problem it solves compared to running an application directly on a machine.
- Explain what a Docker image and a Docker container are, and how they relate to each other.
- Explain what "content management" means in terms of the commands explored.
- Describe the purpose of each command explored: `docker --version`, `docker pull`, `docker images`, `docker run`, `docker ps`, `docker ps -a`, `docker exec`, `docker cp`, `docker inspect`, `docker logs`, `docker stop`, `docker start`, `docker rm`, `docker rmi`.
- Confirm that an image (`ubuntu:24.04`) was actually pulled and verified locally.

This experiment is scoped to Docker content-management commands only. Dockerfiles, Docker Compose, image publishing, and orchestration are not part of this experiment.

Requirements

- Windows with WSL2 enabled and Docker Desktop installed.
- Docker version **29.6.2** (as used in this experiment).
- A terminal with Docker CLI access (via WSL2).

Starting environment note: Before this experiment's work began, the Docker environment already had other project work present — **4 containers (3 running, 1 stopped)** and **21 images**. These belong to other project work on the same machine and were not created for this experiment. They are noted here because they were visible in the output of the commands used to check container/image state.

Concept

Why Docker Exists

An application usually depends on a particular set of software, libraries, and configuration. When it is developed on one machine and then run on another, differences

in that underlying setup can cause the application to behave differently or fail, even though the application's code hasn't changed — often summarized as "it works on my machine, but not on theirs."

Docker lets an application be packaged together with the environment it needs — as a self-contained, isolated unit — so it can run consistently regardless of what else is installed on the underlying machine.

Docker Image vs. Docker Container

- **Docker Image** — a read-only template containing an application and everything it needs to run. Obtained from a registry such as Docker Hub (https://hub.docker.com).
- **Docker Container** — a running (or stopped) instance created *from* an image. A container has its own writable layer on top of the image, so changes made inside it don't alter the original image.

```
Docker Image
      ↓
Docker Container
```

An image is the template; a container is what you get when that template is actually run.

What "Content Management" Means

Concern	Relevant command(s)
Obtaining an image	`docker pull`
Viewing images	`docker images`
Creating/running a container	`docker run`
Viewing container state	`docker ps`, `docker ps -a`
Accessing a running container	`docker exec`
Copying content	`docker cp`
Inspecting Docker objects	`docker inspect`
Viewing container logs	`docker logs`
Stopping/starting containers	`docker stop`, `docker start`
Removing containers	`docker rm`
Removing images	`docker rmi`

Procedure

> ***Note on execution status:*** *Steps 1–5 were actually performed in this experiment with real recorded output. Steps 6, 7, and 9–12 are marked* ***"Explored conceptually"*** *— they document what each command does and when it is used, but no specific invocation or container name was recorded. Step 8 was partially carried out:* `content.txt` *in this experiment's directory is a real artifact from exploring content transfer, but the exact container name used was not recorded. When working through this experiment, try each conceptual command against the* `ubuntu:24.04` *container you create in Step 6.*

Step 1 — Check the Docker Version

Status: Actually performed.

```
docker --version
```

Observe: Docker reported version **29.6.2** in this environment.

Step 2 — View the Existing Local Images

Status: Actually performed.

```
docker images
```

Observe: 21 images were already present locally, from other project work on this machine — not images created by this experiment. Note the count *before* the pull in Step 4, so you can confirm exactly one new image was added afterward.

Step 3 — View the Existing Containers

Status: Actually performed.

`docker ps` alone only shows running containers; `docker ps -a` is needed to see the full picture, including stopped ones.

```
docker ps
docker ps -a
```

Observe: 4 containers existed at this point — **3 running, 1 stopped** — including (among others) `ad-agency-postgres`, `ad-agency-dev-control-plane`, and `petclinic-dev-control-plane`. These belong to other project work on the same machine, not to this experiment.

Step 4 — Pull the Ubuntu 24.04 Image

Status: Actually performed.

```
docker pull ubuntu:24.04
```

Observe: The pull completed successfully. Docker reported download progress on the order of **119 MB / 31.7 MB** — the first number is the uncompressed size; the second is the compressed size actually transferred (Docker transmits images in compressed layers and decompresses them locally). The resulting local image was:

```
Repository: ubuntu
Tag:        24.04
Image ID:   33ceb71981b6...
```

Step 5 — Confirm the Pulled Image Locally

Status: Actually performed.

```
docker images
```

Observe: The `ubuntu:24.04` image, with image ID beginning `33ceb71981b6`, now appears in the local image list alongside the 21 that were already present.

Step 6 — `docker run`

Status: Explored conceptually — no specific execution captured in this experiment's record.

`docker run` creates and starts a new container from a local image. This is how an image (a static template) becomes a container (a live instance).

```
docker run -it --name ubuntu-demo ubuntu:24.04 bash
```

When working through this experiment yourself, run this command against the `ubuntu:24.04` image you pulled in Step 4.

Step 7 — `docker exec`

Status: Explored conceptually — no specific execution captured in this experiment's record.

`docker exec` runs a command inside an already-running container, most commonly to open a shell into it.

```
docker exec -it <container> bash
```

When working through this experiment, run this command against the `ubuntu-demo` container you started in Step 6 — replace `<container>` with `ubuntu-demo`.

Step 8 — `docker cp`

Status: Explored conceptually, with one supporting artifact.

`docker cp` copies files or directories between the host machine and a container's filesystem, in either direction.

```
# Copy content.txt from the host into a running container
docker cp content.txt <container-name>:/tmp/content.txt

# Verify it arrived inside the container
docker exec <container-name> cat /tmp/content.txt

# Copy it back out again
docker cp <container-name>:/tmp/content.txt ./content_from_container.txt
```

Replace `<container-name>` with the name or ID of your running container (visible in `docker ps`).

This experiment's directory contains `content.txt`, whose content is an artifact from exploring content transfer between the host and a container:

```
Docker Content Management Experiment
Updated from Docker host
```

The exact container name used in this experiment's own session was not recorded, so the commands above use a placeholder.

Step 9 — `docker inspect`

Status: Explored conceptually — no specific execution captured in this experiment's record.

`docker inspect` returns detailed, low-level metadata about an image or container — configuration, filesystem layers, and other settings.

```
docker inspect <image-or-container>
```

When working through this experiment, run `docker inspect ubuntu-demo` against your running container and `docker inspect ubuntu:24.04` against the image you pulled — read through the JSON output to see what each field represents.

Step 10 — `docker logs`

Status: Explored conceptually — no specific execution captured in this experiment's record.

`docker logs` displays the output a container's main process has produced. This is how a container's activity is reviewed without needing to `exec` into it.

```
docker logs <container>
```

When working through this experiment, run `docker logs ubuntu-demo` against the container you started in Step 6.

Step 11 — `docker stop` / `docker start`

Status: Explored conceptually — no specific execution captured in this experiment's record.

`docker stop` halts a running container; `docker start` restarts a stopped one without recreating it.

```
docker stop <container>
docker start <container>
```

When working through this experiment, stop and then restart the `ubuntu-demo` container — run `docker ps` after each command to observe the status change.

Step 12 — `docker rm` / `docker rmi`

Status: Explored conceptually — no specific execution captured in this experiment's record.

`docker rm` removes a stopped container; `docker rmi` removes an image. A running container must be stopped before it can be removed, and an image still used by a container cannot be removed until that container is gone.

```
docker rm <container>
docker rmi <image>
```

When working through this experiment, stop `ubuntu-demo` first, remove the container with `docker rm`, then remove the `ubuntu:24.04` image with `docker rmi`.

Verification

Check	Where	Confirms
Docker CLI available	`docker --version`	Reported version 29.6.2
Starting image count	`docker images`	21 images present before the pull
Starting container state	`docker ps` / `docker ps -a`	4 containers total — 3 running, 1 stopped
Ubuntu image pulled	`docker pull ubuntu:24.04`	Completed successfully, ~119 MB / 31.7 MB reported

Check	Where	Confirms
Pulled image present locally	`docker images`	`ubuntu`, tag `24.04`, image ID beginning `33ceb71981b6`
Host↔container content transfer	*(concept, with artifact)*	`content.txt` in this experiment's directory reflects work on this topic

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
A container you know exists isn't in `docker ps`	`docker ps` shows only running containers; stopped ones are hidden	`docker ps -a` shows all containers including stopped	`docker start <container>` to restart a stopped one
`docker rm <container>` fails: "container is running"	Running containers must be stopped before they can be removed	`docker stop <container>`, then `docker rm <container>`	`docker ps -a` — the container should no longer appear
`docker rmi <image>` fails: "image is being used by a stopped container"	Even a stopped container holds a reference to its image; the image cannot be removed until that container is gone	`docker ps -a` to find the container; `docker rm <container>`, then retry `docker rmi <image>`	`docker images` — the image should no longer appear
Your new container or image doesn't stand out in the list	Pre-existing containers/images from other work fill the output	Record counts before starting (`docker ps -a`, `docker images`); your new work appears as additions	Look for your container or image by the specific name or tag you assigned

Quick Reference

Command	Purpose
`docker --version`	Check the installed Docker CLI version
`docker pull <image>`	Download an image from a registry
`docker images`	List locally stored images
`docker run <image>`	Create and start a container from an image

Command	Purpose
`docker ps`	List running containers
`docker ps -a`	List all containers, including stopped ones
`docker exec -it <container> <cmd>`	Run a command inside a running container
`docker cp <src> <dest>`	Copy content between host and container
`docker inspect <target>`	View detailed metadata about an image or container
`docker logs <container>`	View a container's output
`docker stop <container>`	Stop a running container
`docker start <container>`	Start a stopped container
`docker rm <container>`	Remove a container
`docker rmi <image>`	Remove an image

Result

The Docker environment (Docker Desktop with WSL2 on Windows, version 29.6.2) already contained 4 containers (3 running, 1 stopped) and 21 images from other project work, confirmed via `docker ps -a` and `docker images` before any new work began. The Ubuntu 24.04 image was then actually pulled with `docker pull ubuntu:24.04`, completing successfully and adding `ubuntu:24.04` (image ID beginning `33ceb71981b6`) to the local image list, confirmed by re-running `docker images`. The remaining content-management commands were explored at the concept level as documented above.

Experiment 07 — Build and Run an Application Using a Dockerfile, Then Create a New Image Version After Modifying the Application

Aim

To containerize an application using a Dockerfile, build and run it as a Docker image/container, then modify the application and build a new image version to reflect that change.

Learning Objectives

- Explain what a Dockerfile is and how it turns application source into a Docker image.
- Explain why this experiment's application is served using Nginx.
- Build a Docker image from a Dockerfile and run a container from it.
- Explain why an already-built image does not automatically reflect a later source change.
- Build a new, separately tagged image version after modifying the application, and run it as a new container.
- Verify two different versions of the same application, served from two different containers on different ports.

Requirements

- Docker (as set up and explored in Experiment 06 — Docker Desktop with WSL2).
- The application source (`index.html`, `style.css`, `script.js`) and a `Dockerfile`, present in `experiment-07/`.
- A web browser, to verify the running application.

Concept

What a Dockerfile Is

A **Dockerfile** is a plain text file that Docker reads to build an image automatically — it describes what base environment to start from and what application files to include. Without a Dockerfile, a person would have to manually install a web server and copy files every time a new image is needed.

This experiment's Dockerfile:

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/
COPY style.css /usr/share/nginx/html/
COPY script.js /usr/share/nginx/html/
```

- `FROM nginx:alpine` starts the image from an existing, pre-built Nginx image rather than building a web server from scratch.
- Each `COPY` instruction places one of the application's files into Nginx's default folder for served content (`/usr/share/nginx/html/`).

Running `docker build` against this file produces a Docker **image** — a fixed snapshot containing Nginx plus this application's files, ready to be run.

Why Nginx

Nginx is used here as the web server that serves the application's static files (`index.html`, `style.css`, `script.js`) once the container is running. The application is a static HTML/CSS/JS page with no server-side logic, so a plain web server is all that's needed to make it reachable over HTTP.

Why Changing the Application Does Not Change an Existing Image

An image is a snapshot taken at build time. Editing `index.html` afterward changes only the file on disk — it does **not** change any image already built from it, and it does **not** change any container already running from that image. The only way for the change to reach an image is to run `docker build` again, producing a new image.

Image Versioning with Tags

Each image build is given a version **tag** — `v1`, then later `v2` — as part of its name (`techfest-app:v1`, `techfest-app:v2`). This keeps the two builds distinguishable and lets both exist locally at the same time.

Overall Flow

Initial build and run:

```
Application Source → Dockerfile → docker build → techfest-app:v1 → docker run → techfest-container
→ Application served by Nginx
```

After modifying the application:

```
Modify Application → docker build → techfest-app:v2 → docker run → techfest-container-v2 → Updated
Application on port 8081
```

Project Structure

```
experiment-07/
├── Dockerfile
├── index.html
├── style.css
├── script.js
└── README.md
```

Procedure

Step 1 — Prepare the Application and Dockerfile

Place the application files (`index.html`, `style.css`, `script.js`) and the `Dockerfile` together in `experiment-07/`. `docker build` needs the Dockerfile and the files it references (via `COPY`) to be available together in the same build context.

Observe: `experiment-07/` contains `Dockerfile`, `index.html`, `style.css`, and `script.js`.

Step 2 — Build `techfest-app:v1`

```
docker build -t techfest-app:v1 .
```

Observe: The build completes, and `techfest-app:v1` appears in the local image list (`docker images`).

Step 3 — Run `techfest-container`

```
docker run -d --name techfest-container -p <host-port>:80 techfest-app:v1
```

Replace `<host-port>` with any available port on the host machine (the specific port used when this experiment was originally run was not recorded). Check for available ports with `sudo lsof -i :<port>` before running.

Flag	Meaning
`-d`	Detached mode — runs the container in the background
`--name techfest-container`	Assigns a human-readable name for later commands
`-p <host-port>:80`	Maps the chosen host port to port 80 inside the container, where Nginx listens

Observe: `techfest-container` appears as a running container (`docker ps`).

Step 4 — Verify the Application Through Nginx

Open `http://localhost:<host-port>` in a browser, using the port chosen in Step 3.

Observe: The registration page loads, served by Nginx from inside the container.

Step 5 — Modify the Application

Open `experiment-07/index.html` and find the main heading line:

```
<h1>TechFest 2026 - Event Registration</h1>
```

Change it to:

```
<h1>TechFest 2026 - Docker Containerized Application</h1>
```

Save the file.

Observe: This modification is already reflected in the current `index.html` in `experiment-07/`.

Step 6 — Build `techfest-app:v2`

The running `techfest-container` (from `v1`) does not pick up this change on its own — a new image must be built from the updated source.

```
docker build -t techfest-app:v2 .
```

Observe: The build completes, and `techfest-app:v2` appears in the local image list alongside `techfest-app:v1`.

Step 7 — Run `techfest-container-v2`

Map the new container to port 8081 so it can run alongside `techfest-container` without a port conflict.

```
docker run -d --name techfest-container-v2 -p 8081:80 techfest-app:v2
```

Observe: `techfest-container-v2` appears as a running container (`docker ps`), separate from `techfest-container`.

Step 8 — Verify Port 8081 Mapping

Open `http://localhost:8081` in a browser.

Observe: The application loads via port 8081.

Step 9 — Verify the Updated Application Is Being Served

Inspect the loaded page's heading at `http://localhost:8081`.

Observe: The heading reads "TechFest 2026 - Docker Containerized Application," matching the modified `index.html`, confirming `techfest-container-v2` is serving the updated application.

Verification

Check	Where	Confirms
`techfest-app:v1` exists	`docker images`	The first image was built successfully
`techfest-container` runs	`docker ps`	A container was started from `techfest-app:v1`
Application is served (v1)	Browser, via `techfest-container` on the chosen host port	Nginx is serving the application from the `v1` image
Application source modified	`experiment-07/index.html`	Heading changed to "TechFest 2026 - Docker Containerized Application"
`techfest-app:v2` exists	`docker images`	A second image was built after the modification
`techfest-container-v2` runs	`docker ps`	A new container was started from `techfest-app:v2`
Port 8081 serves the updated application	Browser, `http://localhost:8081`	`techfest-container-v2` correctly serves the modified application

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
The running container still shows the old heading after editing `index.html`	A container is an isolated snapshot — it does not re-read host files after it starts	Build a new image first: `docker build -t techfest-app:v2 .`, then run a new container from it	Open `http://localhost:8081` — the updated heading should appear
Building with the same tag silently overwrites the previous image	Docker tags are labels; reusing a tag replaces what it pointed to with no warning	Use distinct tags (`v1` , `v2`) for each build; `docker images` should show both entries	Rebuild with the new tag and verify both images appear in `docker images`
	The host port chosen is already bound	Use port 8081 for `techfest-container-v2` as shown; or	

Problem	Why it happens	What to check/do	Retry / Next
`docker run` fails: "port is already allocated"	by another container or process	`sudo lsof -i :<port>` to identify and free the conflicting port	`docker run -d --name techfest-container-v2 -p 8081:80 techfest-app:v2`

Quick Reference

Command	Purpose
`docker build -t techfest-app:v1 .`	Build the first image version from the Dockerfile
`docker run -d --name techfest-container -p <host-port>:80 techfest-app:v1`	Run a container from the v1 image
`docker build -t techfest-app:v2 .`	Build a new image version after modifying the application
`docker run -d --name techfest-container-v2 -p 8081:80 techfest-app:v2`	Run a new container from the v2 image on port 8081
`docker images`	Confirm both `techfest-app:v1` and `techfest-app:v2` exist locally
`docker ps`	Confirm both containers are running

Result

The application from earlier experiments was containerized using a Dockerfile based on `nginx:alpine`, and built into `techfest-app:v1`. A container, `techfest-container`, was run from that image, and the application was verified as being served through Nginx. The application source was then modified — the page heading was changed to "TechFest 2026 - Docker Containerized Application" — and, because the running `v1` container does not reflect source changes on its own, a new image, `techfest-app:v2`, was built from the updated source. A new container, `techfest-container-v2`, was run from that image on port 8081, and the updated application was verified as correctly served at `http://localhost:8081`.

Experiment 08 — Deploy the Dockerized Application to Kubernetes

Aim

To deploy the Docker image built in Experiment 07 (`techfest-app:v2`) into a local Kubernetes cluster, using a Deployment and a NodePort Service.

Learning Objectives

- Explain why Kubernetes is introduced after Docker, and what problem it solves that Docker alone does not.
- Explain the relationship between a Docker image, a Pod, and a container running inside that Pod.
- Explain what a Kubernetes Deployment is, and why it is used instead of managing a Pod directly.
- Explain the role of `replicas`, `labels`, and `selectors` in a Deployment.
- Explain what a Kubernetes Service is, why it's needed, and the difference between `port` and `targetPort`.
- Explain what a `NodePort` Service does.
- Explain why `imagePullPolicy: Never` matters when using a local image with a local cluster.
- Read `experiment-08/deployment.yaml` and `experiment-08/service.yaml` and explain what each field configures.

This experiment reuses the image built in Experiment 07 (`techfest-app:v2`) rather than building a new one.

Requirements

- The Docker image `techfest-app:v2`, built in Experiment 07.
- A local Kubernetes cluster. This experiment's manifests (`imagePullPolicy: Never`, no registry reference) are written for a **local, single-node cluster where the image is already available to the node** — the pattern used by Kind (Kubernetes in Docker).
- `kubectl`, configured against that cluster.

Concept

Why Kubernetes Comes After Docker

Experiment 07 produced a Docker image and ran it as a single container with `docker run`. That works for one container on one machine, but it doesn't handle: keeping a container running if it crashes, running more than one copy of it, or giving it a stable way to be reached. **Kubernetes** manages *how containers built from an image are run*, rather than running them by hand.

Docker Image vs. Kubernetes Pod

A Docker **image** (`techfest-app:v2`) is still the same static template from Experiment 07. In Kubernetes, that image isn't run directly — it's run inside a **Pod**, which is the smallest unit Kubernetes manages. A Pod wraps one or more containers and is what Kubernetes actually schedules, monitors, and restarts if needed.

What a Deployment Is, and Why Not Just a Pod

A **Deployment** is a Kubernetes object that describes the *desired state* of a set of Pods — which image to run, how many copies, and how to identify them — and continuously works to keep the real state matching that description. Managing a Pod directly means if it dies, it's simply gone; a Deployment notices and creates a replacement automatically.

Replicas

`replicas: 1` tells the Deployment to keep exactly one Pod running from this template. Even with one replica, using a Deployment gets the benefit of Kubernetes recreating the Pod if it fails.

Labels and Selectors

- The Deployment's Pod template gives each Pod it creates the label `app: techfest-app`.
- The Deployment's `selector.matchLabels` (`app: techfest-app`) tells the Deployment which Pods belong to it — it must match the Pod template's labels.
- The Service uses the same label as *its* selector — this is how the Service finds which Pods to send traffic to. Labels and selectors are the mechanism connecting the Service to the Deployment's Pods; they aren't linked by name.

What a Service Is, and Why It's Needed

A Pod's own address can change if it's recreated. A Kubernetes **Service** gives a stable way to reach whichever Pod(s) currently match its selector, without needing to track individual Pod addresses.

`port` vs. `targetPort`

- `targetPort: 80` is the port the *container* is actually listening on (Nginx, same as Experiment 07).
- `port: 80` is the port the *Service itself* exposes inside the cluster.

What NodePort Means

`type: NodePort` makes the Service additionally reachable from outside the cluster, on a port opened on the cluster node itself. Kubernetes assigns this NodePort unless one is explicitly specified — this manifest does not specify one, so the actual assigned NodePort is determined by the cluster at apply time.

Why `imagePullPolicy: Never`

By default, Kubernetes tries to *pull* an image from a registry. `techfest-app:v2` was built locally in Experiment 07 and was never pushed to any registry. `imagePullPolicy: Never` tells Kubernetes not to attempt a pull, and instead expect the image to already exist on the node. In a Kind setup, this means the image must be loaded into the Kind node before the Pod can start.

Architecture

```
techfest-app:v2
    ↓
Deployment: techfest-app (replicas: 1)
    ↓
Pod
    ↓
containerPort: 80 (Nginx)
    ↓
Service: techfest-service
    ↓
NodePort (cluster-assigned)
    ↓
Application
```

Manifests

`experiment-08/deployment.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: techfest-app
spec:
  replicas: 1
  selector:
```

```
    matchLabels:
      app: techfest-app
  template:
    metadata:
      labels:
        app: techfest-app
    spec:
      containers:
        - name: techfest-app
          image: techfest-app:v2
          imagePullPolicy: Never
          ports:
            - containerPort: 80
```

Field	Value	Meaning
`metadata.name`	`techfest-app`	Name of the Deployment object
`spec.replicas`	`1`	One Pod is kept running
`spec.selector.matchLabels`	`app: techfest-app`	How the Deployment identifies Pods as its own
`template.metadata.labels`	`app: techfest-app`	Label applied to Pods this Deployment creates
`containers[0].image`	`techfest-app:v2`	The Experiment 07 image being deployed
`containers[0].imagePullPolicy`	`Never`	Never attempt to pull from a registry
`containers[0].ports[0].containerPort`	`80`	Port Nginx listens on inside the container

`experiment-08/service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: techfest-service
spec:
  selector:
    app: techfest-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: NodePort
```

Field	Value	Meaning
`metadata.name`	`techfest-service`	Name of the Service object
`spec.selector`	`app: techfest-app`	Selects Pods with this label
`ports[0].port`	`80`	Port the Service exposes inside the cluster
`ports[0].targetPort`	`80`	Port on the Pod/container that traffic is forwarded to
`spec.type`	`NodePort`	Exposes the Service on a port on the cluster node

Prerequisites — Kind Cluster Setup

This experiment uses **Kind** (Kubernetes in Docker) as the local cluster tool.

Create the Kind cluster:

```
kind create cluster --name experiment-08
```

Confirm the cluster is running:

```
kubectl get nodes
```

Expected output:

```
NAME                          STATUS   ROLES           AGE
experiment-08-control-plane   Ready    control-plane   ...
```

Load the image into the Kind node:

```
kind load docker-image techfest-app:v2 --name experiment-08
```

Kind runs its node as a Docker container with its own image store, separate from the host's Docker daemon. `kind load docker-image` makes `techfest-app:v2` available inside the cluster node, which is required because `imagePullPolicy: Never` is set.

Procedure

> ***Before You Start:*** *The Prerequisites section above (Kind cluster creation and `kind load docker-image`) must be complete before running any `kubectl` command below. All steps in this Procedure must be performed by the student — "no captured output" means a recording from this experiment's own session is not available, not that the step can be skipped.*

Step 1 — Confirm `techfest-app:v2` Is Available

Status: Conceptual (no captured output).

Confirm the image built in Experiment 07 exists locally and has been loaded into the Kind node (via `kind load docker-image`). Because `imagePullPolicy: Never` is set, the Pod will fail to start if the image isn't already present.

```
docker images
```

Observe: `techfest-app:v2` appears in the local image list. If it is missing, return to Experiment 07 and rebuild the image before continuing.

Step 2 — Apply `deployment.yaml`

Status: Conceptual (command implied by the experiment's stated purpose; exact execution output not captured).

```
kubectl apply -f deployment.yaml
```

Step 3 — Apply `service.yaml`

Status: Conceptual (no captured output).

```
kubectl apply -f service.yaml
```

Step 4 — Check the Deployment and Pod

Status: Conceptual (no captured output).

```
kubectl get deployments
kubectl get pods
```

Observe: The `techfest-app` Deployment shows `1/1` in the READY column. A Pod named `techfest-app-<hash>` shows `Running` status. Note this Pod name — Experiment 09 refers to it by name when demonstrating self-healing.

Step 5 — Check the Service

Status: Conceptual (no captured output).

The actual NodePort is assigned by Kubernetes at apply time and can only be known by checking the running Service.

```
kubectl get service techfest-service
```

Observe: `techfest-service` appears as type `NodePort`. The PORT(S) column shows a mapping such as `80:<NodePort>/TCP`. Note the NodePort number — you need it to access the application in Step 6.

Step 6 — Access the Application

Status: Conceptual (no captured output).

Open `http://<node-address>:<node-port>` in a browser, using the NodePort from Step 5.

Verification

Check	Evidence	Confirms
Deployment `techfest-app`, 1 replica, `techfest-app:v2`	`experiment-08/deployment.yaml`	Deployment is correctly configured
`imagePullPolicy: Never` set	`experiment-08/deployment.yaml`	Cluster uses the already-local image
Pod label `app: techfest-app` matches Deployment selector	`experiment-08/deployment.yaml`	Deployment will recognize its own Pods
Service selector `app: techfest-app` matches Pod label	`experiment-08/service.yaml`	Service will correctly route to the Deployment's Pods
Service `targetPort: 80` matches container's `containerPort: 80`	Both manifests	Traffic reaches the port Nginx listens on
Service `type: NodePort`	`experiment-08/service.yaml`	Service is configured for external access
Deployment/Pod actually running	*(runtime evidence — Experiment 09 Step 1)*	Experiment 09's baseline check shows `techfest-app` Deployment at `1/1`, Pod `techfest-app-6c98cc6db8-bzlvx` Running, on node `experiment-08-control-plane`

Check	Evidence	Confirms
Service assigned a working NodePort	*(runtime evidence — Experiment 09 Step 1)*	Experiment 09's baseline shows `techfest-service` with NodePort `80:30576/TCP`, Cluster-IP `10.96.184.50`

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
Pod stuck in `ImagePullBackOff` or `ErrImageNeverPull`	`imagePullPolicy: Never` means Kubernetes won't pull from a registry; if the image isn't on the Kind node, there is nothing to run	`docker images` to confirm `techfest-app:v2` exists locally; then `kind load docker-image techfest-app:v2 --name experiment-08` to load it into the node	`kubectl get pods` — the Pod should reach `Running` within 30 seconds
`kubectl get deployments` shows `0/1` READY	`spec.selector.matchLabels` doesn't match the Pod template's `labels` — Kubernetes can't identify its own Pods	In `deployment.yaml`, confirm `spec.selector.matchLabels` and `template.metadata.labels` are both `app: techfest-app`	`kubectl apply -f deployment.yaml`; `kubectl get pods` should show `1/1 Running`
Service has no endpoints; application unreachable on the NodePort	Service `selector` doesn't match the Pod labels — no Pod is registered as a backend	In `service.yaml`, confirm `spec.selector` is `app: techfest-app` — same as the Pod label in `deployment.yaml`	`kubectl apply -f service.yaml`; `kubectl get endpoints techfest-service` should show an IP, not `<none>`
Service is reachable but the application doesn't respond	`targetPort` doesn't match the port Nginx listens on inside the container	Both `port` and `targetPort` in `service.yaml` must be `80` — matching `containerPort: 80` in `deployment.yaml`	`kubectl apply -f service.yaml`; retry the NodePort URL
Opening `http://localhost:80` doesn't load the application	The NodePort is a high-numbered port (30000–32767), not port 80	`kubectl get service techfest-service` — the PORT(S) column shows the actual NodePort, e.g., `80:30576/TCP`; use	Open `http://localhost:<NodePort>` in a browser

Problem	Why it happens	What to check/do	Retry / Next
		the number after the colon	

Result

The Docker image `techfest-app:v2`, built in Experiment 07, was configured for deployment to a local Kubernetes cluster using a Deployment named `techfest-app` (1 replica, `imagePullPolicy: Never`, `containerPort: 80`) and a `NodePort` Service named `techfest-service` (selecting `app: techfest-app`, `port: 80` → `targetPort: 80`), as defined in `experiment-08/deployment.yaml` and `experiment-08/service.yaml`. These manifests are consistent — the Service's selector matches the Deployment's Pod labels, and its `targetPort` matches the container's `containerPort`. Actual cluster application and runtime evidence are provided by Experiment 09's Step 1 baseline check.

Experiment 09 — Automate the Process of Running the Containerized Application Using Kubernetes

Aim

To use Kubernetes to automate the running of the Experiment 07 containerized application (`techfest-app:v2`), by reusing the Deployment defined in Experiment 08 and observing how Kubernetes maintains that Deployment's desired state automatically.

Learning Objectives

- Explain the difference between running a container manually (`docker run`) and having Kubernetes manage that container's execution.
- Explain what "desired state" means in Kubernetes, and how a Deployment describes it.
- Explain how a Deployment automates Pod management, including the role of `replicas`.
- Explain Kubernetes's self-healing behaviour — what a Deployment does when a Pod it manages disappears.
- Use `kubectl` to observe whether the cluster's actual state matches a Deployment's desired state.

Requirements

- The Docker image `techfest-app:v2` (Experiment 07).
- The Kubernetes manifests from Experiment 08 — `experiment-08/deployment.yaml` and `experiment-08/service.yaml`. Experiment 09 does not define new manifests; it reuses these as-is.
- A local Kubernetes cluster with `kubectl` configured, as described in Experiment 08.

Prerequisite: Complete Experiment 08 first. The `techfest-app` Deployment and `techfest-service` Service must already be applied to the cluster and the `techfest-app:v2` image must be loaded into the Kind node.

Note on current status: All four steps below were actually performed and verified against a running cluster. The Deployment and Service from Experiment 08 were already present (5 days old at the time of this experiment). Experiment 09's work consisted of checking their state, deliberately deleting the running Pod, and observing Kubernetes recreate it automatically.

Concept

Docker Container Execution vs. Kubernetes-Managed Execution

- **Docker container execution** (`docker run ...`) is a one-time, imperative action: start this container, from this image, now. Once it's running, Docker doesn't do anything further to keep it that way.
- **Kubernetes-managed execution** is declarative: a Deployment describes what *should* be running, and Kubernetes continuously works to make the actual cluster match that description — including restarting things if they stop matching it.

Desired State

"Desired state" is what a Deployment's spec declares should exist — for `techfest-app`, that is exactly one Pod running the `techfest-app:v2` image (`replicas: 1`). Kubernetes keeps comparing the real state of the cluster against this description on an ongoing basis.

Self-Healing — Recreation of Pods

If a Pod managed by a Deployment is deleted (or crashes), the Deployment notices that the actual Pod count no longer matches the desired count and creates a replacement Pod automatically — without anyone running `docker run` or `kubectl create` again. This replacement is a **new** Pod with a new name, not the original one restarted.

How Kubernetes Maintains Desired State

Kubernetes repeatedly checks "does the current state match the desired state?" and, whenever it doesn't, takes action to close that gap. This runs continuously, not just once at deployment time.

Architecture / Flow

```
techfest-app:v2 (Experiment 07 image)
        ↓
Deployment: techfest-app (Experiment 08 manifest, reused)
        ↓
Kubernetes maintains desired state (replicas: 1)
        ↓
Pod is deleted/crashes  →  actual state no longer matches desired state
        ↓
Deployment automatically creates a replacement Pod
        ↓
Desired state (1 running Pod) restored
```

Procedure

Central demonstration:

```
Pod before deletion:   techfest-app-6c98cc6db8-bzlvx
        ↓  (kubectl delete pod)
Pod after deletion:    techfest-app-6c98cc6db8-fcms2   1/1 Running
```

Step 1 — Baseline Check

Status: Actually performed.

Check the cluster node, the `techfest-app` Deployment, its Pod, and the `techfest-service` Service before making any change. This establishes the actual starting state.

```
kubectl get nodes
kubectl get deployments
kubectl get pods
kubectl get services
```

Observe:

```
NODE
experiment-08-control-plane   Ready   control-plane   v1.30.0

DEPLOYMENT
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
techfest-app  1/1     1            1           5d

POD
NAME                            READY   STATUS    RESTARTS      AGE
techfest-app-6c98cc6db8-bzlvx   1/1     Running   1 (4d ago)    4d23h

SERVICE
NAME              TYPE       CLUSTER-IP     PORT(S)        AGE
techfest-service  NodePort   10.96.184.50   80:30576/TCP   5d
```

Verification: The node is `Ready`, the Deployment is at `1/1`, the existing Pod (`techfest-app-6c98cc6db8-bzlvx`) is `Running`, and `techfest-service` is a NodePort Service on `80:30576/TCP`.

Step 2 — Delete the Running Pod to Trigger Self-Healing

Status: Actually performed.

Deleting the Pod removes it from the cluster's actual state, which then no longer matches the Deployment's desired state of 1 running replica.

```
kubectl delete pod techfest-app-6c98cc6db8-bzlvx
```

> ***Note:*** `techfest-app-6c98cc6db8-bzlvx` *is the Pod name from this experiment's recorded run. When you run Step 1, your Pod will have a* ***different name*** *generated by Kubernetes — substitute your actual Pod name from the Step 1 output. Do not copy this name literally.*

Observe:

```
pod "techfest-app-6c98cc6db8-bzlvx" deleted
```

Step 3 — Confirm Kubernetes Automatically Recreated the Pod

Status: Actually performed.

```
kubectl get pods
```

Observe:

```
NAME                            READY   STATUS    RESTARTS   AGE
techfest-app-6c98cc6db8-fcms2   1/1     Running   0          5s
```

A **new** Pod, `techfest-app-6c98cc6db8-fcms2` (a different name from the deleted `techfest-app-6c98cc6db8-bzlvx`), was already `Running` and `1/1` only 5 seconds after the deletion, with no manual command creating it.

Step 4 — Confirm the Deployment's Desired State Is Restored

Status: Actually performed.

```
kubectl get deployments
kubectl get pods
```

Observe:

```
DEPLOYMENT
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
techfest-app   1/1     1            1           5d

POD
NAME                            READY   STATUS    RESTARTS   AGE
techfest-app-6c98cc6db8-fcms2   1/1     Running   0          41s
```

The Deployment remained at `1/1`, and the replacement Pod was still `Running` at 41 seconds old, with zero restarts — confirming a clean, stable recreation.

What This Result Demonstrates

- The `techfest-app` Deployment's desired state is 1 replica — confirmed both before and after the disruption.
- The original Pod, `techfest-app-6c98cc6db8-bzlvx`, was intentionally deleted.
- Kubernetes automatically created a replacement Pod, `techfest-app-6c98cc6db8-fcms2`, without any manual create/run command.
- The replacement Pod reached `1/1 Running` within seconds.
- The Deployment remained at `1/1` throughout.

Verification

Check	Evidence type	Confirms	Status
`techfest-app` Deployment sets `replicas: 1`	Configuration (`experiment-08/deployment.yaml`)	Desired state is declared as 1 running Pod	Established
Node Ready; Deployment `1/1`; Pod `bzlvx` Running; Service NodePort `80:30576/TCP`	Runtime (Step 1)	Baseline desired-state match	Actually performed
`pod "techfest-app-6c98cc6db8-bzlvx" deleted`	Runtime (Step 2)	Original Pod intentionally removed	Actually performed
New Pod `fcms2`, `1/1 Running`, 5s old	Runtime (Step 3)	Kubernetes automatically created a replacement	Actually performed
Deployment `1/1`; Pod `fcms2` Running, 0 restarts, 41s old	Runtime (Step 4)	Desired state restored and stable	Actually performed

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
Replacement Pod stuck in `ImagePullBackOff` after deletion	`techfest-app:v2` was never loaded into the Kind node — the image is unavailable to the cluster	`docker images` to confirm the image exists locally; `kind load docker-image techfest-app:v2 --name experiment-08` if it wasn't loaded	`kubectl get pods` — the replacement Pod should reach `Running`

Problem	Why it happens	What to check/do	Retry / Next
`kubectl get pods` still shows a Pod with the same-looking name after deletion	The replacement is a new Pod with a different random suffix; if it looks identical, the delete command may not have executed	Compare the suffix in the Pod name and check the AGE column — the replacement will be only seconds old	Run `kubectl get pods` again and compare names and ages carefully
`kubectl delete pod <name>` returns "pod not found"	The Pod name is stale — Kubernetes already replaced the Pod from a previous deletion or restart	`kubectl get pods` to get the current Pod name; copy the exact name including the random suffix	`kubectl delete pod <current-pod-name>`

Quick Reference

Command	Purpose
`kubectl get nodes`	Check the cluster node's status
`kubectl get deployments`	Check the Deployment's desired vs. actual replica count
`kubectl get pods`	List current Pods and their names/status
`kubectl get services`	Check the Service's type, cluster IP, and exposed port
`kubectl delete pod <pod-name>`	Deliberately remove the running Pod, to test self-healing

Result

Kubernetes automation of the Experiment 07 containerized application was demonstrated and verified. Using the `techfest-app` Deployment and `techfest-service` Service already established in Experiment 08, a baseline check confirmed the Deployment at `1/1` with its Pod `techfest-app-6c98cc6db8-bzlvx` running.

That Pod was then intentionally deleted, reporting `pod "techfest-app-6c98cc6db8-bzlvx" deleted`. Immediately afterward, `kubectl get pods` showed Kubernetes had automatically created a replacement Pod, `techfest-app-6c98cc6db8-fcms2`, already `1/1 Running` at 5 seconds old. A final check confirmed the Deployment remained at `1/1` and the replacement Pod was still `1/1 Running` with zero restarts at 41 seconds old — actual, observed evidence that Kubernetes automatically maintains a Deployment's desired state without manual intervention.

Experiment 10 — Selenium Automated Testing

Objective

To install and use Selenium WebDriver for automated browser testing, first with a basic browser automation check, and then to automate and verify submission of the TechFest event registration form (from Experiments 01/07).

Learning Objectives

- Explain what Selenium WebDriver is and why it is used for browser automation.
- Set up a Python virtual environment and install Selenium.
- Use the `debuggerAddress` approach to connect Selenium to an already-running Chromium instance.
- Automate a basic page-title check with `test_browser.py`.
- Automate the full registration form submission with `test_techfest.py` and verify the result.

Requirements

- Python 3.12 (installed on WSL/Linux).
- Chromium and ChromeDriver installed on WSL/Linux (same version — version mismatch causes ChromeDriver to fail).
- Selenium 4.47.0, installed via pip into a virtual environment.

Install Chromium and ChromeDriver:

```
sudo apt update
sudo apt install chromium-browser chromium-chromedriver
```

Verify both are present and at the same version:

```
chromium-browser --version
chromedriver --version
```

Both should report the same version (e.g., `151.0.7922.108`).

Concept

What Is Selenium

Selenium is an open-source framework for automating web browsers. It allows a Python script to control a real browser and interact with a web page the same way a person would: opening a URL, filling in form fields, clicking buttons, and reading back what the page displays.

What Is Selenium WebDriver

Selenium WebDriver is the specific part of Selenium used here — the API (`from selenium import webdriver`) that a program uses to create and control a browser session. `webdriver.Chrome(options=options)` creates this controlled browser session.

How Selenium Interacts with Chromium Through ChromeDriver

Selenium WebDriver does not talk to a browser directly. It sends commands to **ChromeDriver**, a separate program that bridges Selenium and Chromium: ChromeDriver receives each command (open this URL, find this element, click it, read its text) and carries it out in the actual browser, then reports the result back.

```
Python test
     ↓
Selenium WebDriver
     ↓
ChromeDriver
     ↓
Chromium
     ↓
Web application
```

Why Selenium Is Used Here

Earlier experiments produced a real, working web application — the TechFest registration form. Verifying it by hand is not repeatable without doing it again each time. Selenium automates exactly that sequence of actions and checks the result programmatically.

Environment / Setup

A Python virtual environment is created at `experiment-10/.venv`, with Selenium 4.47.0 installed into it. The entire setup runs on the **WSL/Linux side**: Chromium (version 151.0.7922.108) and a matching ChromeDriver (151.0.7922.108) are both installed on WSL/Linux.

```
cd experiment-10
python3 -m venv .venv
source .venv/bin/activate
pip install selenium==4.47.0
```

`.venv/` is excluded from version control via `experiment-10/.gitignore`. Experiments 11 and 12 reuse this virtual environment via `source ../experiment-10/.venv/bin/activate`.

Our `debuggerAddress` Approach

Both scripts use:

```
options.add_experimental_option("debuggerAddress", "127.0.0.1:9222")
```

instead of having Selenium start and manage its own browser process. This means:

- A Chromium instance must already be running with remote debugging enabled on port `9222` **before** either script is run — the scripts attach to that existing session rather than launching a new one.
- `driver.quit()` ends the WebDriver session but does not necessarily close a browser it didn't launch.

Project Structure

```
experiment-10/
├── test_browser.py
├── test_techfest.py
├── .gitignore
└── README.md
```

Source Code

`test_browser.py`

A basic Selenium connectivity check — confirms Selenium can attach to the running Chromium session and control it before attempting the form-automation test.

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_experimental_option(
    "debuggerAddress",
    "127.0.0.1:9222"
)
```

```
driver = webdriver.Chrome(options=options)

driver.get("https://example.com")

print("Title:", driver.title)

driver.quit()
```

test_techfest.py

Automates the full TechFest registration form at `http://localhost:8082`.

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import Select

options = Options()
options.add_experimental_option(
    "debuggerAddress",
    "127.0.0.1:9222"
)

driver = webdriver.Chrome(options=options)

driver.get("http://localhost:8082")

driver.find_element("id", "name").send_keys("Test Student")
driver.find_element("id", "email").send_keys("test@example.com")
driver.find_element("id", "phone").send_keys("9876543210")

Select(driver.find_element("id", "department")).select_by_value("cse")
Select(driver.find_element("id", "year")).select_by_value("3")
Select(driver.find_element("id", "event")).select_by_value("web-development")

driver.find_element("css selector", "button[type='submit']").click()

message = driver.find_element("id", "message").text

print("Registration message:", message)

assert message == "Registration successful!"

print("Selenium test passed successfully.")

driver.quit()
```

Procedure

Step 1 — Activate the virtual environment:

```
source .venv/bin/activate
```

Step 2 — Start Chromium with remote debugging enabled (in a separate terminal — keep it running):

```
chromium-browser --remote-debugging-port=9222 &
```

If Chromium crashes or fails to start, add `--no-sandbox` to the command — this flag may be needed depending on the WSL2 kernel configuration.

Step 3 — For `test_techfest.py`, ensure the TechFest registration app is being served at `http://localhost:8082`.

Option A — Docker container (uses the Experiment 07 image):

```
docker run -d -p 8082:80 techfest-app:v2
```

Option B — Python's built-in HTTP server (no Docker needed):

```
cd experiment-01
python3 -m http.server 8082
```

Step 4 — Run the scripts:

```
python test_browser.py
python test_techfest.py
```

Verification / Results

`test_browser.py` — successful output:

```
Title: Example Domain
```

`test_techfest.py` — successful output:

```
Registration message: Registration successful!
Selenium test passed successfully.
```

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
`WebDriverException: unable to connect`	Chromium is not running with remote debugging;	Start Chromium first: `chromium-browser --remote-debugging-`	Re-run `test_browser.py` or `test_techfest.py`

Problem	Why it happens	What to check/do	Retry / Next
when running the script	the scripts attach to an existing session rather than launching their own	`port=9222 &` (in a separate terminal); keep it running while the script runs	
Chromium crashes immediately or shows a sandbox error	WSL2 kernel restricts the default Chromium sandbox	Add `--no-sandbox`: `chromium-browser --remote-debugging-port=9222 --no-sandbox &`	Re-run the test script once Chromium starts successfully
`ModuleNotFoundError: No module named 'selenium'`	The virtual environment was not activated before running the script	`source .venv/bin/activate` from inside `experiment-10/`; the prompt should show `(.venv)`	Re-run the test script
`test_techfest.py` fails — form not loading or elements not found	The TechFest app is not running at `http://localhost:8082`	Start it via Docker: `docker run -d -p 8082:80 techfest-app:v2`, or via Python: `cd experiment-01 && python3 -m http.server 8082`	Re-run `test_techfest.py`
ChromeDriver version mismatch error on startup	Installed versions of `chromium-browser` and `chromium-chromedriver` differ	`chromium-browser --version` and `chromedriver --version` must show the same version; reinstall both: `sudo apt install chromium-browser chromium-chromedriver`	Re-run the test script

Conclusion

Selenium 4.47.0 was installed and used, via Selenium WebDriver and ChromeDriver, to automate two browser test scenarios against a running Chromium instance (151.0.7922.108) on WSL/Linux, connected via `127.0.0.1:9222`: a basic page-title check against `https://example.com`, and an end-to-end form submission check against the TechFest registration app at `http://localhost:8082`. Both scripts executed successfully.

Experiment 11 — JavaScript Calculator Selenium Automated Testing

Objective

To use Selenium WebDriver to automate and verify a simple JavaScript calculator application — entering two numbers, clicking Add, and asserting the computed result is correct.

Learning Objectives

- Reuse the Selenium environment from Experiment 10.
- Automate a JavaScript calculator application via the browser UI.
- Understand why the assertion compares a string (`"30"`) rather than an integer (`30`).

Requirements

- The Experiment 10 virtual environment (`experiment-10/.venv`), with Selenium 4.47.0 installed.
- Chromium (151.0.7922.108) and ChromeDriver (151.0.7922.108) on WSL/Linux (same as Experiment 10).

Concept

Why Selenium Is Used Here

The calculator performs its addition entirely in client-side JavaScript (`addNumbers()` in `script.js`) — there is no server-side logic to test directly. Selenium drives the actual browser UI — entering values, clicking the button, and reading the displayed result — so the test verifies the real, rendered behaviour of the page.

How Selenium Interacts with Chromium Through ChromeDriver

The same communication chain as Experiment 10:

```
Python test → Selenium WebDriver → ChromeDriver → Chromium → Web application
```

Environment / Setup

This experiment reuses the Python virtual environment from Experiment 10 (`experiment-10/.venv`). Chromium and ChromeDriver are the same setup used in Experiment 10.

As in Experiment 10, `test_calculator.py` connects to an **already-running** Chromium instance via `debuggerAddress` at `127.0.0.1:9222`.

The calculator application is served at `http://localhost:8000`.

Project Structure

```
experiment-11/
├── index.html
├── script.js
├── test_calculator.py
└── README.md
```

Source Code

`index.html`

```
<label for="firstNumber">First Number:</label>
<input type="number" id="firstNumber">

<label for="secondNumber">Second Number:</label>
<input type="number" id="secondNumber">

<button id="addButton" onclick="addNumbers()">Add</button>

<p>Result: <span id="result"></span></p>
```

`script.js`

```
function addNumbers() {
    const firstNumber = Number(document.getElementById("firstNumber").value);
    const secondNumber = Number(document.getElementById("secondNumber").value);

    const result = firstNumber + secondNumber;

    document.getElementById("result").textContent = result;
}
```

`test_calculator.py`

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_experimental_option(
    "debuggerAddress",
    "127.0.0.1:9222"
)

driver = webdriver.Chrome(options=options)

driver.get("http://localhost:8000")

driver.find_element("id", "firstNumber").send_keys("10")
driver.find_element("id", "secondNumber").send_keys("20")

driver.find_element("id", "addButton").click()

result = driver.find_element("id", "result").text

print("Calculator result:", result)

assert result == "30"

print("Selenium test passed successfully.")

driver.quit()
```

Note on the string comparison: `element.text` in Selenium always returns a Python `str`, not a number. Even though the calculator computes a numeric result (30), the DOM renders it as text. This is why the assertion is `result == "30"` (a string), not `result == 30` (an integer). If you accidentally compare against an integer, the assertion will fail even when the page is showing the correct value.

Procedure

Step 1 — Activate the Experiment 10 virtual environment (run from inside `experiment-11/`):

```
source ../experiment-10/.venv/bin/activate
```

Step 2 — Start Chromium with remote debugging enabled (in a separate terminal — keep it running):

```
chromium-browser --remote-debugging-port=9222 &
```

If Chromium crashes or fails to start, add `--no-sandbox` to the command.

Step 3 — Serve the calculator application at `http://localhost:8000` (in a separate terminal, from inside `experiment-11/`):

```
python3 -m http.server 8000
```

Keep this server running while the test runs.

Step 4 — Run the test:

```
python test_calculator.py
```

Verification / Results

`test_calculator.py` — successful output:

```
Calculator result: 30
Selenium test passed successfully.
```

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
`AssertionError` even though the page shows `30`	The assertion compares against an integer (`30`) instead of a string — `element.text` always returns a Python `str`	In `test_calculator.py`, confirm the assertion is `assert result == "30"` (with quotes, not `== 30`)	Re-run `test_calculator.py`
`WebDriverException: unable to connect`	Chromium is not running with remote debugging on port 9222	`chromium-browser --remote-debugging-port=9222 &` in a separate terminal; keep it running	Re-run `test_calculator.py`
Script errors trying to find elements — `driver.get("http://localhost:8000")` loads a connection error page	The calculator app is not being served at port 8000	From inside `experiment-11/`, run `python3 -m http.server 8000` in a separate terminal	Re-run `test_calculator.py`
`ModuleNotFoundError: No module named 'selenium'`	Running without activating the	`source ../experiment-10/.venv/bin/activate`	Re-run `test_calculator.py`

Problem	Why it happens	What to check/do	Retry / Next
	Experiment 10 virtual environment	(run from inside `experiment-11/`)	

Conclusion

Selenium 4.47.0, reused from the Experiment 10 virtual environment, was used via Selenium WebDriver and ChromeDriver to automate the JavaScript calculator application on a running Chromium instance (151.0.7922.108) on WSL/Linux, connected via `127.0.0.1:9222`. The test entered `10` and `20`, clicked `addButton`, and asserted the `#result` element showed `"30"`. The script executed successfully, printing `Calculator result: 30` and `Selenium test passed successfully.`

Experiment 12 — Develop Test Cases for the Containerized Application Using Selenium

Objective

To develop and execute Selenium test cases against the containerized TechFest 2026 registration application (deployed in Experiment 07/08), covering valid registration, input filtering behaviour, and required-field validation.

Learning Objectives

- Develop multiple structured test cases against a containerized web application.
- Understand how client-side JavaScript validation (name/phone filtering, required fields) behaves under automated testing.
- Explain why TC04 checks that the `#message` element stays empty when required fields are left blank.

Requirements

- The Experiment 10 virtual environment (`experiment-10/.venv`), with Selenium 4.47.0 installed.
- Chromium (151.0.7922.108) and ChromeDriver (151.0.7922.108) on WSL/Linux (same as Experiments 10 and 11).
- The `techfest-container-v2` Docker container (image `techfest-app:v2`) running and accessible at `http://localhost:8081`.

Concept

Why Selenium Is Used Here

The containerized TechFest application is a real, running web page — its form validation (name/phone filtering) happens in client-side JavaScript, and its success message is only set after a real form submission. Selenium drives the actual rendered page so each test case verifies the container's real, running behaviour rather than the application's source code in isolation.

How Selenium Interacts with Chromium Through ChromeDriver

```
Python test → Selenium WebDriver → ChromeDriver → Chromium → Containerized TechFest application
```

Application Under Test

Detail	Value
Container name	`techfest-container-v2`
Image	`techfest-app:v2`
Application URL	`http://localhost:8081`
Server	`nginx/1.31.4`

Reachability was confirmed with:

```
curl -I http://localhost:8081
```

which returned `HTTP/1.1 200 OK`.

Relevant Application Behaviour

Field	Identifier
Name	`id="name"`
Email	`id="email"`
Phone	`id="phone"`
Department	`id="department"` (select)
Year	`id="year"` (select)
Event	`id="event"` (select)
Submit button	`button[type="submit"]`
Result message	`id="message"`

Client-side JavaScript behaviour: - The **name** input removes any character that is not `A-Z`, `a-z`, or a space. - The **phone** input removes any character that is not a digit. - On successful form submission, the message becomes `"Registration successful!"`.

Environment / Setup

This experiment reuses the Python virtual environment from Experiment 10. Chromium is already running with remote debugging at `127.0.0.1:9222`. `test_techfest_cases.py` connects to this **already-running** session rather than launching its own browser.

Project Structure

```
experiment-12/
├── test_techfest_cases.py
└── README.md
```

Source Code — `test_techfest_cases.py`

The file defines a shared `create_driver()` helper, four test functions, and calls all four directly at the end. No test framework such as pytest is used — the functions are plain Python functions invoked directly.

```python
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import Select


def create_driver():
    options = Options()
    options.add_experimental_option(
        "debuggerAddress",
        "127.0.0.1:9222"
    )
    return webdriver.Chrome(options=options)


def test_valid_registration():
    driver = create_driver()

    driver.get("http://localhost:8081")

    driver.find_element("id", "name").send_keys("Test Student")
    driver.find_element("id", "email").send_keys("test@example.com")
    driver.find_element("id", "phone").send_keys("9876543210")

    Select(driver.find_element("id", "department")).select_by_value("cse")
    Select(driver.find_element("id", "year")).select_by_value("3")
    Select(driver.find_element("id", "event")).select_by_value("web-development")

    driver.find_element("css selector", "button[type='submit']").click()

    message = driver.find_element("id", "message").text

    print("TC01 - Valid registration:", message)

    assert message == "Registration successful!"

    driver.quit()


def test_name_input_filtering():
    driver = create_driver()
```

```
    driver.get("http://localhost:8081")

    name = driver.find_element("id", "name")
    name.send_keys("Test123@ Student!")

    actual_value = name.get_attribute("value")

    print("TC02 - Name after filtering:", actual_value)

    assert actual_value == "Test Student"

    driver.quit()


def test_phone_input_filtering():
    driver = create_driver()

    driver.get("http://localhost:8081")

    phone = driver.find_element("id", "phone")
    phone.send_keys("987abc654@321")

    actual_value = phone.get_attribute("value")

    print("TC03 - Phone after filtering:", actual_value)

    assert actual_value == "987654321"

    driver.quit()


def test_required_fields():
    driver = create_driver()

    driver.get("http://localhost:8081")

    driver.find_element("css selector", "button[type='submit']").click()

    message = driver.find_element("id", "message").text

    print("TC04 - Message with empty required fields:", message)

    assert message == ""

    driver.quit()


test_valid_registration()
test_name_input_filtering()
test_phone_input_filtering()
test_required_fields()

print("All Selenium test cases passed successfully.")
```

Test Cases

TC01 — Valid Registration

Opens `http://localhost:8081`, fills in all fields with valid data (`name=Test Student`, `email=test@example.com`, `phone=9876543210`, `department=cse`, `year=3`, `event=web-development`), submits the form, and verifies `#message` equals `"Registration successful!"`.

TC02 — Name Input Filtering

Enters `Test123@ Student!` into the `name` field and verifies the resulting field value equals `"Test Student"` — confirming characters other than letters and spaces are removed.

TC03 — Phone Input Filtering

Enters `987abc654@321` into the `phone` field and verifies the resulting field value equals `"987654321"` — confirming non-digit characters are removed.

TC04 — Required Fields

Opens the application, clicks the submit button with all required fields left empty, and verifies `#message` remains empty.

Why `message` stays empty: The form's input fields have the HTML5 `required` attribute. When a required field is empty and the submit button is clicked, the browser's built-in validation fires — it blocks the form's `submit` event from reaching JavaScript at all. Because the `submit` event never fires, `message.textContent` is never set. The `message` element remains `""`, which is what the test asserts.

Procedure

Step 0 — Start the container (if not already running):

```
docker run -d --name techfest-container-v2 -p 8081:80 techfest-app:v2
```

Skip this step if the container is already running (check with `docker ps`).

Step 1 — Activate the Experiment 10 virtual environment (run from inside `experiment-12/`):

```
source ../experiment-10/.venv/bin/activate
```

Step 2 — Confirm the container is reachable:

```
curl -I http://localhost:8081
```

`curl -I` sends an HTTP HEAD request, asking the server to respond with just HTTP headers. A response of `HTTP/1.1 200 OK` confirms Nginx inside the container is running and reachable at port 8081. If this step fails, the test cases will also fail trying to connect.

Step 3 — Start Chromium with remote debugging enabled (in a separate terminal — keep it running):

```
chromium-browser --remote-debugging-port=9222 &
```

If Chromium crashes or fails to start, add `--no-sandbox` to the command — this flag may be needed depending on the WSL2 kernel configuration.

Step 4 — Run the test cases directly with Python (not with pytest — the script calls its test functions directly):

```
python test_techfest_cases.py
```

Verification / Results

`test_techfest_cases.py` — exact verified output:

```
TC01 - Valid registration: Registration successful!
TC02 - Name after filtering: Test Student
TC03 - Phone after filtering: 987654321
TC04 - Message with empty required fields:
All Selenium test cases passed successfully.
```

All four test cases executed successfully against the running `techfest-container-v2` container: a valid registration produced the expected success message, the name and phone fields correctly filtered invalid characters, and submitting with empty required fields correctly left the message blank.

Common Mistakes

Problem	Why it happens	What to check/do	Retry / Next
Selenium can't load `http://localhost:8081` — connection refused	The `techfest-container-v2` container is not running	`docker ps` to check; if absent, `docker run -d --name techfest-container-v2 -p 8081:80 techfest-app:v2`	`curl -I http://localhost:8081` — expect `HTTP/1.1 200 OK`; then re-run the test
TC02 or TC03 assertion fails — field contains	Input field holds a value from a previous test	Each test function calls `driver.get("http://localhost:8081")` to reload the page;	Re-run `test_techfest_cases.py`

Problem	Why it happens	What to check/do	Retry / Next
unexpected characters	run in the same browser session	confirm this line is present at the top of each test function in `test_techfest_cases.py`	
`WebDriverException: unable to connect`	Chromium is not running with remote debugging on port 9222	`chromium-browser --remote-debugging-port=9222 &` in a separate terminal; keep it running while the test runs	Re-run `test_techfest_cases.py`
Tests don't execute when running with `pytest`	The test functions are called directly at the bottom of the script, not via pytest markers	Run the script directly: `python test_techfest_cases.py` (not `pytest`)	All four test functions should execute and print their results

Conclusion

Selenium 4.47.0, reused from the Experiment 10 virtual environment, was used via Selenium WebDriver and ChromeDriver to develop and execute four test cases against the containerized TechFest registration application (`techfest-container-v2`, image `techfest-app:v2`) at `http://localhost:8081`, connected to a running Chromium instance (151.0.7922.108) on WSL/Linux via `127.0.0.1:9222`. All four test cases — valid registration, name input filtering, phone input filtering, and required-field validation — executed successfully, ending with `All Selenium test cases passed successfully.`

End of DevOps Laboratory Manual